



**Politechnika
Śląska**

Dokumentacja Projektowa

Projekt zaliczeniowy

E-SOR - System wspomagania klasyfikacji priorytetów pacjentów na SOR

Kierunek: Informatyka

Stopień: I, Rok: II

Członkowie zespołu:

Sebastian Gomolec

Mateusz Hubczyk

Daniel Kowalski

Spis treści

1. Wstęp i Opis Projektu	4
1.1. Geneza i kontekst dziedzinowy (Triage na SOR)	4
1.2. Cel główny i cele szczegółowe systemu	4
1.3. Ogólny zakres funkcjonalny projektu	4
2. Organizacja pracy i zarządzanie projektem	5
2.1. Metodyka zarządzania zadaniami i komunikacja	5
2.2. Skład zespołu deweloperskiego i podział ról	5
2.3. Przepływ pracy z kodem źródłowym (GitHub & GitFlow)	6
2.4. Harmonogram prac i opis Sprintów	6
2.4.1. Sprint 1 – Faza Inicjalizacji i Analizy	6
2.4.2. Sprint 2 – Rdzeń Systemu i Architektura Bazodanowa	7
2.4.3. Sprint 3 – Integracja AI oraz Testy Systemowe	7
2.4.4. Sprint 4 – Finalizacja i Stabilizacja	7
3. Architektura Infrastrukturalna i DevOps	7
3.1. Specyfikacja sprzętowo-sieciowa serwera produkcyjnego	7
3.2. Koegzystencja z usługami pozaprojektowymi	8
3.3. Przykładowy plik zmiennych środowiskowych .env	9
3.4. Izolacja kontenerowa i konfiguracja środowiskowa (Docker & Docker Compose)	9
3.5. Specyfikacja i funkcje kontenerów w obrębie projektu E-SOR	9
3.6. Model hybrydowego trasowania bazodanowego (Multi-Environment Setup)	10
3.6.1. Integracja i interakcja między kontenerami	10
3.7. Topologia bezpiecznej sieci VPN (Szyfrowanie WireGuard i kontrola dostępu Tailscale)	11
4. Projekt i Implementacja Warstwy Serwerowej (Backend API)	11
4.1. Analiza porównawcza i uzasadnienie wyboru języka Go oraz frameworku Gin	11
4.1.1. Aspekt edukacyjny i rozwój kompetencji (Przesłanka Główna)	11
4.1.2. Reaktywność w czasie rzeczywistym i obsługa protokołu WebSocket	12
4.1.3. Charakterystyka uruchomieniowa (Zgodność z architekturą Homelab)	12
4.1.4. Efektywność frameworku Gin Gonic i integracja z middleware	12
4.2. Autoryzacja	12
4.2.1. Middleware	13
4.2.2. Algorytmy szyfrowania	13
4.2.3. Wykorzystywane biblioteki	13
4.3. Dokumentacja Ścieżek	13
4.3.1. Zapytanie bez tokenu	13
4.3.2. Autoryzacja	14
4.3.2.1. Rejestracja użytkownika	14
4.3.2.2. Logowanie i Generowanie Tokenu JWT	14
4.3.2.3. Pobieranie Profilu Zalogowanego Użytkownika	15
4.3.3. Pacjenci	15
4.3.3.1. Rejestracja nowej karty pacjenta	15
4.3.3.2. Wyszukiwanie pacjenta w systemie	16
4.3.3.3. Zaawansowane filtrowanie i wyszukiwanie pacjentów	17
4.3.3.4. Aktualizacja danych pacjenta	18
4.3.3.5. Usuwanie karty pacjenta	18

4.3.4.	Przyjęcia na SOR	18
4.3.4.1.	Wstępna ocena stanu pacjenta i sugerowany kod pilności (AI / Heurystyka)	18
4.3.4.2.	Zapisanie nowej karty przyjęcia (Triage)	19
4.3.4.3.	Strumieniowanie aktywnej kolejki pacjentów w czasie rzeczywistym (Główny ekran poczekalni)	20
4.3.4.4.	Aktualizacja statusu karty przyjęcia	21
4.3.4.5.	Archiwum wszystkich przyjęć na SOR	22
4.3.4.6.	Pobranie szczegółów pojedynczego przyjęcia	23
4.3.4.7.	Pobieranie historii wszystkich wizyt pacjenta na podstawie numeru PESEL	24
4.3.4.8.	Archiwum i historia wszystkich przyjęć na SOR	24
4.3.4.9.	Anulowanie karty przyjęcia	25
4.3.5.	Panel Lekarza i Diagnostyka	25
4.3.5.1.	Pobranie zgłoszeń przypisanych do zalogowanego lekarza	25
4.3.5.2.	Pobranie historii zgłoszeń lekarza (Zamknięte karty)	26
4.3.5.3.	Przejęcie pacjenta z poczekalni (Wezwanie do gabinetu)	26
4.3.5.4.	Zlecenie badania diagnostycznego	27
4.3.5.5.	Finalizacja wizyty lekarskiej (Konsultacja końcowa)	27
4.3.6.	Moduł Personelu oraz Publiczny Przegląd Kolejki (RODO)	28
4.3.6.1.	Wyszukiwarka i Katalog Personelu	28
4.3.6.2.	Anonimowy Strumień Telewizora Poczekalni	28
4.4.	Środowisko Wytwórcze i Narzędzia Deweloperskie	29
4.4.1.	Zintegrowane Środowisko Programistyczne (IDE)	29
4.4.2.	Lokalna Konteneryzacja i Wirtualizacja Środowiska	29
4.4.3.	Testowanie i Walidacja Punktów Końcowych HTTP (REST)	29
4.4.4.	Testowanie Dwukierunkowych Strumieni WebSocket	29
5.	Moduł Analityczny Sztucznej Inteligencji	30
5.1.	Architektura mikrousługi Python / FastAPI	30
5.1.1.	Opis Architektury	30
5.1.2.	Opis wybranych narzędzi i bibliotek	30
5.2.	Opis matematyczny i teoretyczny wybranego modelu uczenia maszynowego	30
5.2.1.	Struktura Topologiczna Drzewa	31
5.2.2.	Metryka Nieczystości Węzła (Indeks Giniego)	31
5.2.3.	Optymalizacja Podziału i Zysk Informacyjny	31
5.2.4.	Kryteria Stopu i Rekurencja	31
5.2.5.	Proces Predykcji	32
5.3.	Proces inżynierii cech (Feature Engineering) i wektor wejściowy parametrów życiowych	32
5.4.	Proces trenowania modelu, dobór hiperparametrów i zestaw danych uczących	32
5.5.	Analiza metryk wydajnościowych modelu (Accuracy, Precision, Recall, F1-Score)	33
5.6.	Integracja i komunikacja synchroniczna Backend-AI	34
5.7.	Komunikacja z modelem poprzez API	35
6.	Projekt i Implementacja Aplikacji Klientkiej (Frontend)	36
6.1.	Wybór technologii i frameworka	36

6.2.	Architektura aplikacji klienckiej	36
6.3.	Wykorzystane biblioteki i zarządzanie stanem	37
6.4.	Faza projektowania (UX/UI)	38
6.5.	Struktura katalogów i plików (Feature-First)	39
6.6.	Opis głównych modułów i funkcjonalności systemu	40
6.6.1.	Moduł Autoryzacji (Logowanie i kontrola sesji)	40
6.6.2.	Moduł Rejestracji Pacjenta i Wywiadu Triage	41
6.6.3.	Moduł Dashboardu i Aktywnego Kolejowania	42
6.6.4.	Moduł Konsultacji Lekarskiej (Panel Lekarza)	43
6.7.	Zapewnienie Jakości Oprogramowania (Testy Jednostkowe)	44
6.8.	Budowanie, kompilacja i dystrybucja wieloplatformowa	45
7.	Projekt Warstwy Danych i Konfiguracja Relacyjnej Bazy Danych	46
7.1.	Uzasadnienie wyboru bazy danych PostgreSQL	46
7.2.	Architektura mapowania obiektowo-relacyjnego (GORM & Auto-Migracje) . .	46
7.3.	Struktura tabel oraz relacji	46
7.3.1.	Słowniki systemowe (Typy ENUM)	46
7.3.2.	Specyfikacja tabel bazodanowych	47
7.3.2.1.	Tabela: patients (Pacjenci)	47
7.3.2.2.	Tabela: staff (Personel)	47
7.3.2.3.	Tabela: admissions (Przyjęcia i Triage na SOR)	48
7.3.2.4.	Tabela: consultations (Konsultacje Lekarskie)	49
7.3.2.5.	Tabela: diagnostic_orders (Zlecenia Diagnostyczne)	49
7.3.3.	Relacje i więzy integralności referencyjnej	50
7.3.4.	Integralność referencyjna i relacje bazodanowe	50
7.3.5.	Przykładowe zapytania SQL używane w aplikacji	50
7.3.5.1.	1. Pobranie aktywnej kolejki pacjentów (Preload i sortowanie KTAS)	50
7.3.5.2.	2. Pobranie szczegółów pojedynczego przyjęcia po ID	51
7.3.5.3.	3. Filtrowanie archiwum przyjęć na SOR	51
7.3.5.4.	4. Zapisanie nowej karty Triage (Rejestracja)	51
7.3.5.5.	5. Aktualizacja statusu pacjenta (Wezwanie do gabinetu) . . .	51
7.4.	Diagram UML bazy danych	52
7.4.1.	Generowanie Diagramu UML Struktury Bazy Danych	52
7.4.1.1.	Metodologia deklaratywna (Diagram-as-Code)	52
7.4.1.2.	Środowisko i narzędzia renderujące	52
7.5.	Diagram UML	53
8.	Podsumowanie i Wnioski Końcowe	55
8.1.	Stopień realizacji celów inżynierskich	55
8.2.	Wnioski deweloperskie i edukacyjne	55
8.3.	Kierunki dalszego rozwoju systemu	56

1. Wstęp i Opis Projektu

1.1. Geneza i kontekst dziedzinowy (Triage na SOR)

Szpitalne Oddziały Ratunkowe (SOR) stanowią krytyczny element infrastruktury ochrony zdrowia, działający w warunkach ciągłej nieprzewidywalności, wysokiego stresu personelu oraz limitowanych zasobów kadrowo-sprzętowych. Jednym z najpoważniejszych wyzwań współczesnego ratownictwa medycznego jest zjawisko przeludnienia oddziałów (ang. **crowding**), które bezpośrednio koreluje ze wzrostem śmiertelności pacjentów oraz wydłużeniem czasu oczekiwania na pierwszą pomoc.

W celu optymalizacji zarządzania strumieniem pacjentów stosuje się procedurę **Triage** (segregację medyczną). Jej zadaniem jest przypisanie pacjenta do jednej z pięciu kategorii priorytetu pilności (np. według standardów KTAS - **Korean Triage and Acuity Scale** lub Manchester Triage System). W warunkach presji czasu i deficytu informacji, manualna ocena parametrów życiowych dokonywana przez personel medyczny jest podatna na błędy subiektywne.

Niniejszy projekt, zatytułowany **E-SOR**, stanowi odpowiedź na ten problem poprzez wprowadzenie zintegrowanego, reaktywnego ekosystemu informatycznego wspomaganego algorytmami sztucznej inteligencji.

1.2. Cel główny i cele szczegółowe systemu

Celem głównym projektu jest zaprojektowanie, zaimplementowanie i wdrożenie rozproszonego systemu wspomaganego decyzji medycznych, który automatyzuje proces klasyfikacji pacjentów na podstawie parametrów krytycznych (triada przeżycia, skala bólu, wiek) w czasie rzeczywistym.

Do celów szczegółowych (realizowanych w ramach poszczególnych przedmiotów akademickich) należą:

- **Wydaźność i Orkiestracja:** Stworzenie bezstanowej, wysokowydajnej warstwy serwerowej (Backend API) w języku Go, zdolnej do asynchronicznego procesowania żądań i utrzymywania stabilnych połączeń strumieniowych.
- **Predykcja i Analiza AI:** Opracowanie niezależnej mikrouslugi w języku Python realizującej inferencję z wykorzystaniem wytrenowanego modelu uczenia maszynowego do predykcji stopnia pilności KTAS.
- **Mobilność i Ergonomia UI/UX:** Zaimplementowanie responsywnej, natywnej/hybrydowej aplikacji klienckiej (Frontend) zapewniającej intuicyjną pracę personelu w dynamicznym środowisku szpitalnym.
- **Architektura DevOps i Bezpieczeństwo:** Konteneryzacja całego środowiska, implementacja automatycznego procesu ciągłego wdrażania (CI/CD) z zachowaniem pełnej izolacji wrażliwych danych medycznych przy użyciu szyfrowanej sieci kratowej VPN.

1.3. Ogólny zakres funkcjonalny projektu

System w swojej architekturze zamyka pełen cykl obsługi pacjenta na SOR:

1. **Rejestracja i Wyszukiwanie:** Szybkie wprowadzanie danych metrykalnych pacjenta lub przeszukiwanie bazy danych po unikalnych identyfikatorach (PESEL).
2. **Automatyczny Triage AI:** Pobranie parametrów życiowych przez ratownika, automatyczna analiza przez moduł AI i zwrot sugerowanego kodu kolorystycznego (Czerwony, Pomarańczowy, Żółty, Zielony, Niebieski).
3. **Dynamiczne Kolejowanie:** Automatyczne układanie kolejki pacjentów w poczekalni na podstawie priorytetu medycznego oraz realnego czasu oczekiwania.

4. **Panel Gabinetowy:** Narzędzia dla lekarzy do przejmowania pacjentów z kolejki, tworzenia zleceń diagnostycznych (RTG, krew) oraz wpisywania finalnych diagnoz.
5. **Prezentacja Publiczna:** Anonimowy strumień danych dla pacjentów wyświetlany na ekranach zbiorczych w poczekalni (Kiosk/TV).

2. Organizacja pracy i zarządzanie projektem

W celu zapewnienia wysokiej efektywności, terminowości oraz transparentności procesów wytwórczych podczas realizacji systemu E-SOR, zespół deweloperski wdrożył zwinne standardy zarządzania projektami informatycznymi, opierając się na metodyce Scrum z rodziny Agile oraz ujednoliconym przepływie pracy nad kodem źródłowym.

2.1. Metodyka zarządzania zadaniami i komunikacja

Praca nad systemem była zorganizowana w postaci 4 dynamicznych, dwutygodniowych iteracji (Sprintów). Każda z nich była ukierunkowana na dostarczenie konkretnych, działających elementów systemu (kamieni milowych).

Do obsługi procesów projektowych wykorzystano następujące narzędzia:

- **Atlassian Jira:** System ten służył jako centralna platforma do mapowania wymagań funkcjonalnych, śledzenia postępów prac oraz zarządzania backlogiem projektu za pomocą tablicy Scrum. Bieżący status zadań odzwierciedlał cykl życia kodu za pomocą kolumn: *To Do*, *In Progress*, *Code Review* oraz *Done*.
- **Discord:** Komunikator ten stanowił rdzeń codziennej, operacyjnej komunikacji zespołu. Pozwalał na szybką wymianę informacji, bieżące konsultacje architektoniczne oraz synchronizację prac pomiędzy deweloperami backendu, frontendu i modułu AI.
- **Cotygodniowe spotkania synchroniczne (Sync):** Cykliczne statusy zespołu realizowane na koniec każdego tygodnia pozwalały na ewaluację postępów, retrospekcję, a także dekompozycję oraz estymację nowych zadań.

2.2. Skład zespołu deweloperskiego i podział ról

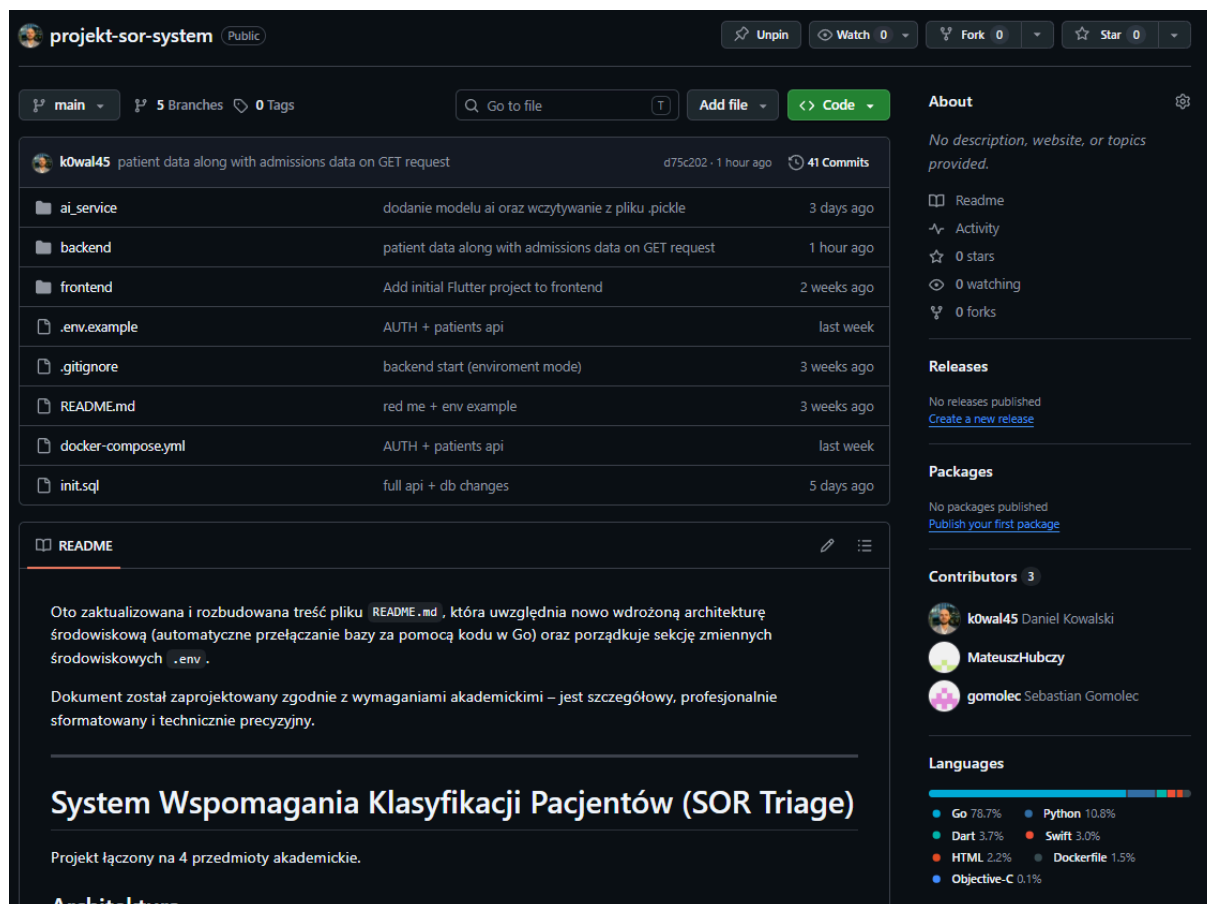
Projekt realizowany był przez zbalansowany, 3-osobowy zespół inżynierski, w którym każdy z członków odpowiadał za dedykowany i odseparowany technologicznie obszar systemu:

1. **Sebastian Gomolec — Frontend Developer:** Odpowiedzialny za projekt, architekturę (wzorzec MVVM) oraz implementację aplikacji klienckiej w technologii **Flutter**. Do jego zadań należało zarządzanie stanem aplikacji, budowa reaktywnego interfejsu użytkownika (UI/UX) dostosowanego do realiów pracy na SOR oraz integracja dwukierunkowych strumieni **WebSocket** obsługujących aktualizacje kolejki w czasie rzeczywistym.
2. **Mateusz Hubczyk — AI Core Developer:** Odpowiedzialny za wytworzenie warstwy analitycznej systemu w języku **Python** z wykorzystaniem frameworka **FastAPI**. Przeprowadził analizę danych wejściowych, zaprojektował i wytrenował model drzewa decyzyjnego klasyfikacji pilności KTAS, a także odpowiadał za serializację modeli do plików produkcyjnych **.pkl** i wystawienie wydajnego interfejsu API mikrousługi AI.
3. **Daniel Kowalski — Backend & DevOps Developer:** Odpowiedzialny za architekturę serwerową, bazodanową oraz infrastrukturę wdrożeniową. Zaprojektował i zaimplementował nadrzędny serwer aplikacji w języku **Go** (**eseor-backend**) z wykorzystaniem frameworka **Gin** i systemu ORM **GORM**. Odpowiadał za strukturę relacyjnej bazy danych **PostgreSQL**,

mechanizmy bezpiecznej autoryzacji opartej o tokeny **JWT** (JSON Web Tokens) oraz pełną konteneryzację środowiska za pomocą narzędzia **Docker Compose**.

2.3. Przepływ pracy z kodem źródłowym (GitHub & GitFlow)

Całość kodu źródłowego projektu była hostowana w rozproszonym systemie kontroli wersji **GitHub**. W celu zachowania porządku w kodzie oraz eliminacji konfliktów deweloperskich wdrożono uproszczony model **GitFlow**



Rysunek 1: Zrzut ekranu repozytorium Github projektu

Prace integracyjne były realizowane manualnie poprzez procedurę **Pull Request (PR)**. Po zakończeniu prac nad daną funkcjonalnością, autor kodu otwierał wniosek o scalenie gałęzi do main. Warunkiem koniecznym do wykonania fuzji kodu było przejście procedury wzajemnej weryfikacji (**Code Review**) i uzyskanie akceptacji od innego członka zespołu.

2.4. Harmonogram prac i opis Sprintów

Prace deweloperskie zostały precyzyjnie rozłożone na cztery główne fazy:

2.4.1. Sprint 1 – Faza Inicjalizacji i Analizy

Pierwsza faza koncentrowała się na przygotowaniu środowiska pracy oraz fundamentów teoretycznych.

- Skonfigurowano repozytoria na platformie GitHub, utworzono projekt i tablice w systemie Jira oraz przeprowadzono szczegółową analizę medycznych danych wejściowych standardu KTAS.

- Rozpisano kompletny backlog zadań.
- Przygotowano wstępne kontenery Docker oraz rozpoczęto proces trenowania pierwszych wersji modeli uczenia maszynowego w Pythonie.

2.4.2. Sprint 2 – Rdzeń Systemu i Architektura Bazodanowa

W tym sprincie zaimplementowano kluczowe komponenty strukturalne aplikacji.

- Daniel Kowalski zaprojektował fizyczny model relacyjny bazy danych PostgreSQL oraz uruchomił backend w Go, implementując podstawowe endpointy operacji CRUD.
- Sebastian Gomolec przygotował architekturę aplikacji Flutter opartą o wzorzec MVVM i powiązał ją z mechanizmem zarządzania stanem.
- Zaimplementowano bezpieczny moduł uwierzytelniania użytkowników oraz autoryzację ról personelu medycznego (lekarz, ratownik, pielęgniarz, admin) wykorzystujący tokeny JWT.

2.4.3. Sprint 3 – Integracja AI oraz Testy Systemowe

Faza ta była kluczowa dla uzyskania pełnej, inteligentnej funkcjonalności systemu E-SOR.

- Mateusz Hubczyk sfinalizował proces uczenia modelu sztucznej inteligencji, generując ostateczne pliki produkcyjne formatu .pkl.
- Przeprowadzono pełną integrację nadrzędnego backendu w Go z mikrousługą analityczną FastAPI AI za pomocą wewnętrznych protokołów sieciowych Dockera.
- Zaimplementowano asynchroniczną komunikację za pomocą protokołu WebSocket (obsługa kolejki w czasie rzeczywistym).
- Przeprowadzono kompleksowe, manualne testy integracyjne end-to-end (E2E) weryfikujące poprawność przepływu danych na linii: interfejs użytkownika Flutter serwer Go model AI Python.

2.4.4. Sprint 4 – Finalizacja i Stabilizacja

Ostatnia iteracja poświęcona była pracom wykończeniowym oraz walidacji.

- Dokonano ostatecznego przeglądu i optymalizacji zapytań SQL warstwy bazodanowej.
- Przeprowadzono pełną walidację i korektę całości technicznej dokumentacji inżynierskiej.
- Opracowano sekcje podsumowujące oraz przygotowano strukturę i materiały na prezentację projektową.

3. Architektura Infrastrukturalna i DevOps

3.1. Specyfikacja sprzętowo-sieciowa serwera produkcyjnego

Środowisko produkcyjne systemu E-SOR zostało uruchomione w architekturze „on-premise” na dedykowanej jednostce klasy Mini-PC, zlokalizowanej w prywatnej sieci domowej jednego z autorów projektu. Wybór tej infrastruktury podyktowany był chęcią zachowania pełnej kontroli nad przetwarzanymi danymi testowymi oraz optymalizacją kosztów utrzymania systemu. Urządzenie działa w trybie ciągłym (24/7), a stabilność połączenia sieciowego oraz minimalizację opóźnień (latency) zabezpiecza bezpośrednie połączenie kablowe z routerem brzegowym.

- **Architektura:** Prywatna sieć domowa (Home Lab), host fizyczny typu Mini-PC
- **Model:** NiPoGi E1
- **Nazwa hosta w systemie:** homeserwer
- **Procesor (CPU):** Intel N97 Alder Lake x86_64 (4 rdzenie, 4 wątki)
- **Pamięć operacyjna (RAM):** 16GB DDR4

- **Dysk (Storage):** 512GB NVMe SSD (system + bazy danych) + 4TB HDD (kopie zapasowe)
- **System operacyjny:** Ubuntu Server 24.04 LTS (wersja bezśrodowiskowa)
- **Sposób wpięcia do sieci:** Kabel Ethernet RJ-45 S/FTP cat. 6 podłączony bezpośrednio do routera, przypisany lokalny stały adres IP (Static DHCP)
- **Producent sprzętu:** Shenzhen CYX Industrial Co., Ltd.

3.2. Koegzystencja z usługami pozaprojektowymi

Wdrożenie systemu E-SOR opiera się w całości na technologii konteneryzacji Docker oraz narzędziu Docker Compose. Wykorzystanie izolacji kontenerowej było kluczowe, ponieważ opisywana maszyna pełni jednocześnie funkcję prywatnego serwera domowego (Home Server). Wszystkie mikroserwisy projektu E-SOR (Backend Go, Baza PostgreSQL, Serwis AI Python) działają w wydzielonej sieci wirtualnej Dockera (**bridge**), dzięki czemu koegzystują w sposób niezakłócony i w pełni bezpieczny z innymi usługami uruchomionymi na tym samym hoście.

Wykaz kluczowych usług działających równolegle na maszynie hosta:

- **Watchtower:** Narzędzie monitorujące i automatycznie aktualizujące obrazy bazowe pozostałych kontenerów w tle.
- **Immich:** Samoobsługowa, wysokowydajna aplikacja do zarządzania i backupu multimediów (zdjęć i filmów).
- **Jellyfin:** Rozproszony, otwartoźródłowy serwer multimedialny do strumieniowania w sieci lokalnej.
- **Serwer Minecraft (Paper/Purpur):** Dedykowany serwer gier wieloosobowych uruchomiony w kontenerze Docker. Z uwagi na specyfikę silnika gry, usługa ta posiada sztywno zaalokowaną pamięć RAM (poprzez flagi JVM `-Xms4G -Xmx4G`), co gwarantuje stabilność rozgrywki bez ryzyka nagłego skoku użycia zasobów maszyny hosta, który mógłby wpłynąć na wydajność bazy danych PostgreSQL lub modułu AI systemu E-SOR.
- **Infrastruktura wspólna:** Wszystkie serwisy współdzielą zasoby procesora oraz pamięci RAM, a izolacja portów sieciowych zapobiega konfliktom z aplikacją E-SOR.

== Topologia architektury mikroservisowej (Separation of Concerns)

System E-SOR został zaimplementowany w architekturze mikroservisowej w celu odizolowania warstwy prezentacji, logiki biznesowej, bazy danych oraz modułu analitycznego AI. Komunikacja pomiędzy komponentami odbywa się za pomocą synchronicznych zapytań HTTP REST (JSON) oraz asynchronicznych strumieni full-duplex (WebSockets).

Projekt dzieli się na następujące domeny odpowiedzialności:

- **Frontend:** Klient mobilny (Flutter/Native) oraz telewizor zbiorczy poczekalni. Odpowiadają wyłącznie za renderowanie UI i konsumpcję API.
- **Backend API (Go / Gin):** Realizuje walidację danych, autoryzację personelu (JWT), modyfikację stanu bazy danych oraz dystrybucję zdarzeń przez WebSockets.
- **AI Core (Python / FastAPI):** Odizolowany serwis predykcyjny. Pobiera dane wejściowe parametrów życiowych, wykonuje inferencję matematyczną z użyciem modelu i zwraca wyliczony priorytet KTAS.
- **Data Layer (PostgreSQL):** Relacyjna baza danych przechowująca dane metrykalne pacjentów, sesje użytkowników oraz historię wpisów medycznych.

3.3. Przykładowy plik zmiennych środowiskowych .env

```
# =====  
# SYSTEM CONFIGURATION (.env)  
# =====  
APP_ENV=development  
IS_SERVER=false  
SERVER_IP=192.168.0.10  
  
# --- GITHUB / GIT-SYNC CONFIGURATION ---  
GIT_USERNAME=username  
GIT_TOKEN=porawnyTokenGithub  
  
# --- POSTGRESQL DATABASE CONFIGURATION ---  
DB_USER=admin  
DB_PASSWORD=password  
DB_NAME=sor_system  
DB_PORT=5432  
  
# Adres URL do mikrousługi AI dla żądań Triage  
AI_SERVICE_URL=http://ai:8000/predict  
  
# JWT  
JWT_SECRET=naszSekretnyKod
```

3.4. Izolacja kontenerowa i konfiguracja środowiskowa (Docker & Docker Compose)

Izolacja środowiskowa na serwerze produkcyjnym realizowana jest za pomocą silnika Docker. Wszystkie usługi systemu E-SOR pracują w odizolowanych kontenerach i są zarządzane przez jedną instancję Docker Compose.

Mechanizmy zabezpieczające przed wpływem usług pozaprojektowych:

1. **Izolacja sieciowa (Docker Bridge Network):** Stos projektowy E-SOR operuje w dedykowanej wirtualnej sieci `sor_network` z osobnym mostkiem i własną domeną rozgłoszeniową. Kontenery pozaprojektowe (np. chmury, DNS) nie mają fizycznego dostępu do tej podsieci.
2. **Izolacja przestrzeni nazw (Namespaces):** Procesy `sor_backend` czy `sor_db` posiadają odizolowane tablice procesów (PID namespaces) i systemy plików (mount namespaces), co uniemożliwia im interakcję z procesami innych usług na hoście.
3. **Unikalność portów:** Ruch zewnętrzny do systemu kierowany jest wyłącznie przez dedykowane porty zmapowane na hoście (np. 8080 dla backendu), wykluczając konflikty z innymi aplikacjami na serwerze.

3.5. Specyfikacja i funkcje kontenerów w obrębie projektu E-SOR

Wszystkie mikroserwisy oraz kontenery pomocnicze wchodzące w skład architektury projektu E-SOR są spięte w jedną domenę zarządzania wewnątrz pliku `docker-compose.yml`. Każdy kontener pełni autonomiczną funkcję w systemie, a komunikacja między nimi jest ściśle kontrolowana za pomocą mapowania portów oraz wewnętrznego DNS sieci wirtualnej.

Poniższa tabela przedstawia kompletną strukturę sieciową oraz odpowiedzialność poszczególnych komponentów środowiska:

Nazwa kontenera	Nazwa obrazu / Źródło	Port (Host)	Zakres odpowiedzialności i rola w systemie
sor_backend	Budowany lokalnie (build: ./backend)	8080/tcp	Główny serwer API (Go/Gin). Odpowiada za routing HTTP, autoryzację personelu przez JWT, logikę biznesową kolejki oraz zarządzanie dwukierunkowymi strumieniami WebSocket dla personelu i ekranu publicznego TV.
sor_ai	Budowany lokalnie (build: ./ai_service)	8001/tcp	Mikrousluga analityczna (Python/FastAPI). Izoluje model uczenia maszynowego. Udostępnia wewnętrzny endpoint, który na żądanie backendu przetwarza wektor parametrów życiowych pacjenta i zwraca wyliczony priorytet KTAS (1–5).
sor_db	PostgreSQL:15 (Oficjalny obraz)	5432/tcp	Relacyjna baza danych. Odpowiada za trwałe i transakcyjne przechowywanie struktur danych zdefiniowanych w modelach GORM (karty pacjentów, dane personelu, historia przyjęć na SOR, rejestr badań).

3.6. Model hybrydowego trasowania bazodanowego (Multi-Environment Setup)

W celu elastycznego zarządzania architekturą bez konieczności modyfikowania pliku `docker-compose.yml`, wdrożono trzytrybowy system routingu, sterowany dynamicznie za pomocą zmiennych środowiskowych `.env` na poziomie jądra aplikacji Go.

Środowisko	Zmienne .env	Charakterystyka i Routing Sieciowy
Development (PC)	APP_ENV=development IS_SERVER=false	Backend działa na komputerze dewelopera i łączy się z lokalnym, deweloperskim kontenerem PostgreSQL (db) w tej samej sieci Dockera.
Hybrydowe (PC)	APP_ENV=production IS_SERVER=false	Backend uruchomiony na komputerze dewelopera. Wykrycie flagi <code>production</code> przy braku <code>IS_SERVER</code> wymusza przekierowanie zapytań SQL na zewnętrzny adres <code>SERVER_IP</code> (serwera domowego), umożliwiając testy na realnych danych bez narzutu na PC.
Produkcja (Serwer)	APP_ENV=production IS_SERVER=true	Aplikacja działa bezpośrednio na serwerze domowym. Ignoruje trasowanie zewnętrzne i łączy się z produkcyjnym kontenerem bazy danych lokalnie po sieci mostkowej Dockera (db).

3.6.1. Integracja i interakcja między kontenerami

Zastosowanie powyższej konfiguracji gwarantuje, że:

1. **Baza danych (sor_db)** jest całkowicie bezpieczna — mimo że nasłuchuje na domyślnym porcie 5432, ruch do niej jest filtrowany przez interfejs sieci kratowej Tailscale i dostęp do niej mają wyłącznie autoryzowane stacje robocze oraz wewnętrzny kontener backendu.
2. **Izolacja kodu AI** zapobiega narzutowi wydajnościowemu na główną aplikację w Go. W przypadku intensywnych obliczeń matematycznych wykonywanych przez model w Pythonie, kontener `sor_ai` zużywa odizolowane zasoby procesora, nie blokując przy tym obsługi żądań HTTP ani połączeń WebSocket realizowanych przez `sor_backend`.
3. **Proces aktualizacji** nie wpływa na ciągłość działania bazy danych. Komenda wywoływana przez `sor_webhook` wymusza przebudowanie wyłącznie kontenerów aplikacyjnych (`backend` oraz `ai`), co zapobiega przerwaniu sesji bazodanowych i chroni przed uszkodzeniem tabel PostgreSQL.

3.7. Topologia bezpiecznej sieci VPN (Szyfrowanie WireGuard i kontrola dostępu Tailscale)

Ponieważ fizyczny serwer `homeserwer` znajduje się za podwójnym mechanizmem NAT operatora ISP i nie posiada publicznego adresu IP, bezpieczna łączność zdalna (dla środowiska hybrydowego na PC oraz aplikacji mobilnych) została zrealizowana w architekturze sieci kratowej (Mesh VPN) za pomocą technologii **Tailscale**.

Specyfikacja warstwy bezpieczeństwa:

- **Kryptografia:** Ruch wewnątrz sieci VPN jest w pełni szyfrowany asymetrycznie przy użyciu protokołu **WireGuard** (szyfry ChaCha20 i Poly1305).
- **Kontrola dostępu (Access Control):** Port bazy danych 5432 oraz porty API nie są wystawione na publiczny ruch WAN. Serwer nasłuchuje wyłącznie na interfejsie sieciowym przypisanym przez Tailscale.
- **Polityka autoryzacji urządzeń:** Każde nowe urządzenie (PC dewelopera, smartfon z aplikacją) próbujące skomunikować się z serwerem pod adresem `100.114.242.128` musi zostać jawnie zatwierdzone w panelu administracyjnym przez administratora sieci (Access Acceptance Policy), co uniemożliwia dostęp osobom nieuprawnionym.

4. Projekt i Implementacja Warstwy Serwerowej (Backend API)

4.1. Analiza porównawcza i uzasadnienie wyboru języka Go oraz frameworku Gin

Decyzja o wyborze stosu technologicznego dla warstwy serwerowej systemu E-SOR opiera się na synergii pomiędzy technicznymi wymaganiami architektury rozproszonej działającej w czasie rzeczywistym a kluczowym celem akademickim projektu, jakim jest rozwój kompetencji inżynierskich zespołu deweloperskiego.

4.1.1. Aspekt edukacyjny i rozwój kompetencji (Przesłanka Główna)

Nadrzędnym argumentem determinującym wybór języka Go (Golang) była świadoma decyzja autorów o wykorzystaniu projektu jako platformy do ekspandowania własnych umiejętności poza ramy standardowego programu studiów. Podjęcie wyzwania polegającego na implementacji systemu medycznego w języku Go – z którym autorzy nie mieli wcześniejszego doświadczenia – pozwoliło na poszerzenie kompetencji zawodowych o paradygmaty silnie typowanych języków kompilowanych, zorientowanych na maksymalną współbieżność.

4.1.2. Reaktywność w czasie rzeczywistym i obsługa protokołu WebSocket

Z punktu widzenia inżynierii oprogramowania, system Triage na SOR-ze musi przetwarzać dane bezprzerwowo i utrzymywać stałe, asynchroniczne połączenia strumieniowe z wieloma urządzeniami klienckimi (ekrany w poczekalni, tablety ratowników, aplikacje mobilne) jednocześnie. Zastąpienie klasycznej architektury REST protokołem WebSocket wymusiło wybór środowiska zdolnego do masowej obsługi otwartych socketów TCP.

Tradycyjne ekosystemy (np. Python z blokadą GIL lub ciężkie wątki systemowe w Java/.NET) generują w takich scenariuszach wysoki narzut pamięciowy i wąskie gardła procesowe. Go rozwiązuje ten problem natywnie poprzez mechanizm **Goroutines** — wirtualnych wątków zarządzanych w przestrzeni użytkownika przez planer (Go Scheduler), których początkowy koszt pamięciowy wynosi zaledwie 2 KB. Pozwala to na jednoczesne orkiestrowanie setek połączeń oraz natychmiastowe rozgłaszanie (*broadcast*) aktualizacji o stanie kolejki pacjentów przy znikomym obciążeniu procesora serwera.

4.1.3. Charakterystyka uruchomieniowa (Zgodność z architekturą Homelab)

Ponieważ system jest wdrażany na fizycznym minikomputerze współdzielonym z wymagającymi usługami domowymi (m.in. serwer multimedialny Jellyfin, system kopii zapasowych Immich czy konteneryzowany serwer gry Minecraft oparty na silniku Java i modyfikacji Forge), kluczowa była rygorystyczna optymalizacja alokacji zasobów.

Go kompiluje się bezpośrednio do natywnego pliku binarnego (kod maszynowy), eliminując potrzebę stosowania interpretatorów czy ciężkich maszyn wirtualnych. W efekcie kontener `sor_backend` cechuje się natychmiastowym czasem rozruchu (*Cold Start* liczony w milisekundach) oraz minimalnym poborem pamięci RAM w stanie spoczynku (4–12 MB). Gwarantuje to pełną stabilność systemu ratunkowego oraz bezkonfliktową koegzystencję z pozostałymi serwisami na maszynie hosta, zapobiegając wyzwalaniu mechanizmów *OOM Killer* systemu Linux.

4.1.4. Efektywność frameworku Gin Gonic i integracja z middleware

JAS jako uzupełnienie wydajności języka Go, do obsługi routingu HTTP oraz uścisku dłoni (Handshake) protokołu WebSocket wybrano framework **Gin Gonic**. Jego silnik oparty na strukturze drzewa prefiksowego (Radix Tree) gwarantuje, że dopasowywanie ścieżek API (np. `/api/ws/admissions/queue`) odbywa się w optymalnym czasie logarytmicznym, całkowicie eliminując zasobożerne operacje oparte na wyrażeniach regularnych.

Dodatkowo, Gin udostępnia wysoce wydajny potok przetwarzania (**Middleware Chain**). Architektura ta pozwoliła na elastyczne rozbudowanie mechanizmu autoryzacji (AuthMiddleware), który w sposób hybrydowy potrafi weryfikować poprawność tokenów JWT — zarówno ze standardowych nagłówek HTTP `Authorization: Bearer`, jak i bezpośrednio z parametrów zapytania URL (Query String), co stało się kluczowym fundamentem zabezpieczenia strumieniowych punktów końcowych systemu E-SOR.

4.2. Autoryzacja

System E-SOR wykorzystuje architekturę uwierzytelniania opartą na tokenach **JWT (JSON Web Tokens)** oraz kontrolę dostępu opartą na rolach personelu medycznego (**RBAC**).

Proces działa bezstanowo: podczas logowania serwer weryfikuje tożsamość użytkownika i zwraca podpisany kryptograficznie token JWT, który klient (np. aplikacja mobilna) musi dołączać do każdego kolejnego zapytania w nagłówku `Authorization`.

4.2.1. Middleware

Middleware działa jako automatyczny „strażnik” umieszczony przed właściwymi ścieżkami API. Jego zadaniem jest filtrowanie i zabezpieczanie ruchu. W systemie proces ten realizują dwa komponenty:

1. AuthMiddleware (Uwierzytelnianie)

Przechwytuje nagłówek `Authorization: Bearer <token>`. Sprawdza poprawność cyfrowego podpisu tokenu oraz jego datę ważności. Jeśli token jest prawidłowy, Middleware wyciąga z niego dane użytkownika (ID oraz rolę) i przekazuje żądanie dalej do systemu. W przeciwnym wypadku natychmiast odrzuca połączenie z błędem `401 Unauthorized`.

2. RoleBlockMiddleware (Kontrola ról)

Odpowiada za logikę RBAC. Sprawdza, czy rola wyciągnięta z tokenu znajduje się na liście ról uprawnionych do wykonania danej akcji. Jeśli np. ratownik medyczny próbuje wywołać funkcję przeznaczoną wyłącznie dla lekarza, to middleware blokuje żądanie i zwraca status `403 Forbidden`.

4.2.2. Algorytmy szyfrowania

W celu zapewnienia poufności i integralności danych system wykorzystuje dwa standardy kryptograficzne:

1. Bcrypt

Służy do bezpiecznego haszowania haseł deweloperów i personelu przed zapisem do bazy danych. Bcrypt automatycznie generuje losową sól (*salt*) i wykonuje wielokrotne, kosztowne obliczeniowo cykle szyfrowania, co uniemożliwia odczytanie oryginalnego hasła nawet w przypadku wycieku bazy danych.

2. HMAC-SHA256 (HS256)

Algorytm używany do generowania podpisu cyfrowego w tokenach JWT. Łączy on zawartość tokenu (dane o użytkowniku) z tajnym kluczem serwera `JWT_SECRET` za pomocą funkcji skrótu SHA-256. Dzięki temu serwer potrafi w ułamku sekundy wykryć, czy użytkownik nie próbował samodzielnie zmodyfikować swojej roli w tokenie.

4.2.3. Wykorzystywane biblioteki

Warstwa bezpieczeństwa została zaimplementowana przy użyciu oficjalnych i sprawdzonych pakietów ekosystemu Go:

- github.com/golang-jwt/jwt/v5 – Odpowiada za pełną obsługę tokenów: ich strukturę, bezpieczne podpisywanie kluczem serwera oraz parsowanie i weryfikację przychodzących żądań.
- golang.org/x/crypto/bcrypt – Dostarcza gotowe, zoptymalizowane funkcje do generowania bezpiecznych skrótów haseł oraz ich późniejszego bezpiecznego porównywania podczas procesu logowania.

4.3. Dokumentacja Ścieżek

Wszystkie żądania do poniższych punktów końcowych muszą być kierowane na adres bazowy serwera z przedrostkiem `/api`. Formatem wymiany danych dla zapytań oraz odpowiedzi jest wyłącznie **JSON**.

4.3.1. Zapytanie bez tokenu

Ścieżka: `GET /api/patients`

Opis: Zapytanie do chronionych przez middleware ścieżek.

```
Authorization: none
```

Schemat odpowiedzi (Response - 401 Unauthorized):

```
{
  "error": "Brak nagłówka Authorization"
}
```

4.3.2. Autoryzacja

4.3.2.1. Rejestracja użytkownika

Ścieżka: POST /api/auth/register

Opis: Tworzy nowe konto pracownika w systemie i haszuje hasło za pomocą algorytmu Bcrypt.

Schemat zapytania (Request Body):

```
{
  "first_name": "Jan",
  "last_name": "Kowalski",
  "academic_title": "dr n. med.",
  "role": "LEKARZ",
  "login_email": "jan.kowalski@esor.pl",
  "password": "super_tajne_haslo_123"
}
```

Schemat odpowiedzi (Response - 201 Created):

```
{
  "message": "Konto personelu zostało pomyślnie utworzone"
}
```

4.3.2.2. Logowanie i Generowanie Tokenu JWT

Ścieżka: POST /api/auth/login

Opis: Weryfikuje dane uwierzytelniające i zwraca token autoryzacyjny ważny przez 24 godziny.

Dopuszczane role: ADMIN

Wymagany nagłówek (Header):

```
Authorization: Bearer <twój_token_jwt>
```

Schemat zapytania (Request Body):

```
{
  "login_email": "jan.kowalski@esor.pl",
  "password": "super_tajne_haslo_123"
}
```

Schemat odpowiedzi (Response - 200 OK):

```
{
  "token": "qwertyuiop...",
  "user": {
    "id": 1,
    "first_name": "Jan",
    "last_name": "Kowalski",
    "role": "LEKARZ"
  }
}
```

4.3.2.3. Pobieranie Profilu Zalogowanego Użytkownika

Ścieżka: GET /api/auth/me

Opis: Zwraca dane aktualnie zalogowanego członka personelu na podstawie przesłanego tokenu. Trasa zabezpieczona przez AuthMiddleware.

Wymagany nagłówek (Header):

```
Authorization: Bearer <twój_token_jwt>
```

Schemat odpowiedzi (Response - 200 OK):

```
{
  "id": 1,
  "first_name": "Jan",
  "last_name": "Kowalski",
  "role": "LEKARZ",
  "email": "jan.kowalski@esor.pl"
}
```

4.3.3. Pacjenci

Wymagany nagłówek (Header):

```
Authorization: Bearer <twój_token_jwt>
```

4.3.3.1. Rejestracja nowej karty pacjenta

Ścieżka: POST /api/patients

Opis: Zakłada nową kartę pacjenta na SOR i zapisuje informacje o jego stanie zdrowia w bazie danych.

Schemat zapytania (Request Body):

```
{
  "pesel": "95061612345",
  "first_name": "Adam",
  "last_name": "Nowak",
  "date_of_birth": "1995-06-16",
  "gender": "M",
  "address": "ul. Dworcowa 12/4, 40-001 Katowice",
  "phone": "+48500600700",
}
```



```

    "email": "adam.nowak@gmail.com",
    "emergency_contact_name": "Anna Nowak (Żona)",
    "emergency_contact_phone": "+48600700800",
    "blood_group": "0+",
    "allergies": "Penicylina, orzechy arachidowe",
    "chronic_diseases": "Nadciśnienie tętnicze"
  }
}

```

Schemat odpowiedzi (Response - 201 Created):

```

{
  "message": "Karta pacjenta została pomyślnie utworzona",
  "patient": {
    "id": 1,
    "pesel": "95061612345",
    "first_name": "Adam",
    "last_name": "Nowak",
    "date_of_birth": "1995-06-16T00:00:00Z",
    "gender": "M",
    "address": "ul. Dworcowa 12/4, 40-001 Katowice",
    "phone": "+48500600700",
    "email": "adam.nowak@gmail.com",
    "emergency_contact_name": "Anna Nowak (Żona)",
    "emergency_contact_phone": "+48600700800",
    "blood_group": "0+",
    "allergies": "Penicylina, orzechy arachidowe",
    "chronic_diseases": "Nadciśnienie tętnicze",
    "created_at": "2026-06-16T15:30:00Z"
  }
}

```

4.3.3.2. Wyszukiwanie pacjenta w systemie

Ścieżka: GET /api/patients/pesel

Opis: Pobiera pełne dane medyczne i kontaktowe pacjenta na podstawie jego numeru PESEL.

Przykładowy adres URL: http://localhost:8080/api/patients/95061612345

Schemat odpowiedzi (Response - 200 OK):

```

{
  "id": 1,
  "pesel": "95061612345",
  "first_name": "Adam",
  "last_name": "Nowak",
  "date_of_birth": "1995-06-16T00:00:00Z",
  "gender": "M",
  "address": "ul. Dworcowa 12/4, 40-001 Katowice",
  "phone": "+48500600700",
  "email": "adam.nowak@gmail.com",
  "emergency_contact_name": "Anna Nowak (Żona)",
  "emergency_contact_phone": "+48600700800",
  "blood_group": "0+",
  "allergies": "Penicylina, orzechy arachidowe",

```

```

    "chronic_diseases": "Nadciśnienie tętnicze",
    "created_at": "2026-06-16T15:30:00Z"
  }
}

```

4.3.3.3. Zaawansowane filtrowanie i wyszukiwanie pacjentów

Ścieżka: GET /api/patients

Opis: Dynamiczne pobieranie listy pacjentów spełniających określone kryteria medyczne, demograficzne lub czasowe. Wszystkie parametry są opcjonalne i można je dowolnie ze sobą łączyć w adresie URL (*Query Parameters*).

Dostępne parametry filtrowania:

- **first_name** – Wyszukiwanie po imieniu (wyszukiwanie częściowe, niezależne od wielkości liter).
- **last_name** – Wyszukiwanie po nazwisku (wyszukiwanie częściowe, niezależne od wielkości liter).
- **gender** – Filtrowanie po płci (wartości: M, K, INNY).
- **blood_group** – Filtrowanie po grupie krwi (np. 0-, A+, AB+).
- **older_than** – Zwraca pacjentów starszych niż podana liczba lat (np. **older_than=65** dla grupy geriatrycznej).
- **younger_than** – Zwraca pacjentów młodszych niż podana liczba lat (np. **younger_than=18** dla grupy pediatrycznej).
- **registered_since** – Filtrowanie po dacie rejestracji na SOR w formacie YYYY-MM-DD (pozwala odfiltrować pacjentów z bieżącego dyżuru).

Przykłady użycia:

1. Wyszukiwanie pacjentów z uniwersalną grupą krwi wprowadzoną od konkretnego dnia:

```
/api/patients?blood_group=0-&registered_since=2026-06-15
```

2. Filtrowanie niepełnoletnich pacjentów o nazwisku zawierającym frazę „Kow”:

```
/api/patients?last_name=Kow&younger_than=18
```

Schemat odpowiedzi (Response - 200 OK):

```

[
  {
    "id": 3,
    "pesel": "20261698765",
    "first_name": "Kamil",
    "last_name": "Kowalski",
    "date_of_birth": "2020-03-10T00:00:00Z",
    "gender": "M",
    "blood_group": "A+",
    "created_at": "2026-06-16T10:00:00Z"
  }
]

```

4.3.3.4. Aktualizacja danych pacjenta

Ścieżka: PUT /api/patients/:pesel

Opis: Modyfikuje wybrane pola w kartotece pacjenta. W ciele żądania wystarczy przesłać jedynie te pola, które wymagają edycji. Zmiana klucza głównego (PESEL) oraz identyfikatora wewnętrznego (ID) jest zablokowana ze względów bezpieczeństwa.

Schemat zapytania (Request Body - np. zmiana adresu i telefonu):

```
{
  "address": "nowy. Adres 45, Katowice",
  "phone": "+48777888999"
}
```

Schemat odpowiedzi (Response - 200 OK):

```
{
  "message": "Dane pacjenta zostały pomyślnie zaktualizowane",
  "patient": {
    "id": 1,
    "pesel": "95061612345",
    "address": "nowy. Adres 45, Katowice",
    "phone": "+48777888999"
  }
}
```

4.3.3.5. Usuwanie karty pacjenta

Ścieżka: DELETE /api/patients/:pesel

Opis: Trwale usuwa rekord pacjenta z bazy danych systemu E-SOR. Operacja ta wymaga najwyższych uprawnień personelu medycznego.

Dopuszczone role: ADMIN, LEKARZ

Schemat odpowiedzi (Response - 200 OK):

```
{
  "message": "Karta pacjenta została trwale usunięta z systemu"
}
```

4.3.4. Przyjęcia na SOR

Wymagany nagłówek (Header) dla wszystkich poniższych endpointów:

```
Authorization: Bearer <twój_token_jwt>
```

4.3.4.1. Wstępna ocena stanu pacjenta i sugerowany kod pilności (AI / Heurystyka)

Ścieżka: POST /api/admissions/predict-ktas

Opis: Punkt końcowy służący do asynchronicznego wyznaczania sugerowanego priorytetu KTAS (w skali 1–5) na podstawie parametrów życiowych i fizjologicznych pacjenta.

Mechanika działania (Integracja Backend-AI):

1. **Ekstrakcja danych i inżynieria cech:** Serwer backendowy w Go przyjmuje żądanie, a następnie na podstawie przekazanego `id_pacjenta` wykonuje zapytanie do bazy danych PostgreSQL, aby pobrać pełną datę urodzenia i obliczyć aktualny wiek pacjenta w latach.
2. **Komunikacja synchroniczna:** Przygotowany (`age`, `hr`, `sbp`, `dbp`, `pain_lvl`) jest pakowany do formatu JSON i przesyłany za pomocą blokującego żądania HTTP POST do odizolowanego kontenera `sor_ai` na endpoint `/api/triage`.
3. **Inferencja i zwrot:** Mikrousluga FastAPI przetwarza wektor wejściowy przez zserializowany model drzewa decyzyjnego (`DecisionTreeClassifierRaw`), po czym zwraca wyliczony priorytet, który backend mapuje na ujednolicony format wyjściowy.

Schemat zapytania (Request Body):

```
{
  "id_pacjenta": 1,
  "forma_przybycia": "Karetka publiczna",
  "injury": true,
  "mental_status": "Nieprzytomny/Brak reakcji",
  "pain": true,
  "pain_lvl": 9,
  "hr": 145,
  "sbp": 85,
  "dbp": 50,
  "rr": 24,
  "bt": 36.2,
  "chief_complaint": "Pacjent po wypadku samochodowym, podejrzenie urazu
wielonarządowego, utrata przytomności."
}
```

Schemat odpowiedzi (Response - 200 OK):

```
{
  "suggested_priority_ktas": 1
}
```

4.3.4.2. Zapisanie nowej karty przyjęcia (Triage)

Ścieżka: POST `/api/admissions`

Opis: Tworzy i zatwierdza ostateczną kartę przyjęcia pacjenta na oddział. Pacjent domyślnie otrzymuje status `W_POCZEKALNI` i zostaje automatycznie przypisany do kolejki SOR.

Schemat zapytania (Request Body):

```
{
  "id_pacjenta": 1,
  "forma_przybycia": "Karetka publiczna",
  "injury": true,
  "mental_status": "Nieprzytomny/Brak reakcji",
  "pain": true,
  "pain_lvl": 9,
  "hr": 145,
  "sbp": 85,
  "dbp": 50,
```

```

    "rr": 24,
    "bt": 36.2,
    "chief_complaint": "Pacjent po wypadku samochodowym, podejrzenie urazu
wielonarządowego, utrata przytomności.",
    "priority_ktas": 1,
    "is_ai_predicted": true
}

```

Schemat odpowiedzi (Response - 201 Created):

```

{
  "message": "Pacjent zarejestrowany na oddziale i dodany do kolejki",
  "admission": {
    "id": 1,
    "id_pacjenta": 1,
    "id_osoby_przyjmujacej": 2,
    "id_lekarza_prowadzacego": null,
    "data_przyjecia": "2026-06-18T10:45:00Z",
    "forma_przybycia": "Karetka publiczna",
    "injury": true,
    "mental_status": "Nieprzytomny/Brak reakcji",
    "pain": true,
    "pain_lvl": 9,
    "hr": 145,
    "sbp": 85,
    "dbp": 50,
    "rr": 24,
    "bt": 36.2,
    "chief_complaint": "Pacjent po wypadku samochodowym, podejrzenie urazu
wielonarządowego, utrata przytomności.",
    "priority_ktas": 1,
    "is_ai_predicted": true,
    "status_przyjecia": "W_POCZEKALNI"
  }
}

```

4.3.4.3. Strumieniowanie aktywnej kolejki pacjentów w czasie rzeczywistym (Główny ekran poczekalni)

Ścieżka: WS -> ws://ADRES_IP_SERWERA:8080/api/ws/admissions/queue?token=TOKEN

Opis: Ustanawia stałe, dwukierunkowe połączenie protokołem WebSocket w celu asynchronicznego strumieniowania kolejki pacjentów ze statusem W_POCZEKALNI. Wyniki są automatycznie sortowane priorytetowo: najpierw od najwyższego stopnia pilności KTAS (od 1 do 5), a w ramach tego samego kodu według czasu oczekiwania. Połączenie automatycznie wypycha zaktualizowany pakiet kompletnych obiektów przyjęć wraz z zagnieżdżonymi danymi osobowymi pacjentów przy każdej modyfikacji bazy danych.

Autoryzacja: Z uwagi na ograniczenia niektórych klientów WebSocket w kwestii przesyłania niestandardowych nagłówków podczas uścisku dłoni (Handshake), token JWT przekazywany jest bezpośrednio w adresie URL jako parametr zapytania: ?token=<czysty_token_jwt>.

Schemat przesyłanej wiadomości (Strumień JSON po ustanowieniu połączenia / Broadcast):

```
[
  {
    "id": 3,
    "id_pacjenta": 3,
    "id_osoby_przyjmujacej": 2,
    "id_lekarza_prowadzacego": null,
    "data_przyjecia": "2026-06-18T18:19:08.094416Z",
    "forma_przybycia": "Pieszo",
    "injury": false,
    "mental_status": "W pełni świadomy",
    "pain": true,
    "pain_lvl": 5,
    "hr": 98,
    "sbp": 130,
    "dbp": 80,
    "rr": 18,
    "bt": 37.1,
    "chief_complaint": "Silny, kłujący ból w klatce piersiowej promieniujący do lewej ręki od około godziny. Pacjent zgłosił się sam.",
    "priority_ktas": 3,
    "is_ai_predicted": false,
    "status_przyjecia": "W_POCZEKALNI",
    "patient": {
      "id": 3,
      "pesel": "88112298765",
      "first_name": "Jan",
      "last_name": "Kowalski",
      "date_of_birth": "1988-11-22T00:00:00Z",
      "gender": "M",
      "address": "ul. Stawowa 5, 40-095 Katowice",
      "phone": "+48600111222",
      "email": "jan.kowalski@wp.pl",
      "emergency_contact_name": "Marek Kowalski (Brat)",
      "emergency_contact_phone": "+48700111222",
      "blood_group": "A-",
      "allergies": "Brak",
      "chronic_diseases": "Cukrzyca typu 2",
      "created_at": "2026-06-18T18:17:10.253809Z"
    }
  }
]
```

4.3.4.4. Aktualizacja statusu karty przyjęcia

Ścieżka: PATCH /api/admissions/:id/status

Opis: Zmienia bieżący status procesu obsługi pacjenta na oddziale ratunkowym (np. wezwanie do gabinetu lekarskiego lub zakończenie wizyty). Wykonanie tej operacji automatycznie wyzwała asynchroniczne powiadomienie roszyłane protokołem WebSocket (BroadcastQueue), dzięki czemu monitory w poczekalni natychmiastowo aktualizują swój stan.

Dozwolone wartości pola status_przyjecia: W_POCZEKALNI, W_GABINECIE, OCZEKUJE_NA_WYNIKI, ZAKONCZONE.

Schemat zapytania (Request Body):

```
{
  "status_przyjecia": "W_GABINECIE"
}
```

Schemat odpowiedzi (Response - 200 OK):

```
{
  "message": "Status przyjęcia został pomyślnie zaktualizowany",
  "status": "W_GABINECIE"
}
```

4.3.4.5. Archiwum wszystkich przyjęć na SOR

Ścieżka: GET /api/admissions

Opis: Zwraca pełną historię zarejestrowanych przyjęć oddziałowych z wbudowanym mechanizmem Preload("Patient"). Każdy element kolekcji zawiera pełen zbiór parametrów parametrycznych oraz zagnieżdżony obiekt danych osobowych przypisanego pacjenta pod kluczem "patient". Posiada opcjonalne parametry filtrowania zasobów (Query Parameters).

Parametry Query (Opcjonalne): status_przyjecia, priority_ktas

Przykładowy URL: http://localhost:8080/api/admissions?status_przyjecia=ZAKONCZONE&priority_ktas=3

Schemat odpowiedzi (Response - 200 OK):

```
[
  {
    "id": 12,
    "id_pacjenta": 1,
    "id_osoby_przyjmujacej": 2,
    "id_lekarza_prowadzacego": 4,
    "data_przyjecia": "2026-06-23T19:40:00Z",
    "forma_przybycia": "Karetka publiczna",
    "injury": true,
    "mental_status": "Stabilny",
    "pain": true,
    "pain_lvl": 5,
    "hr": 92,
    "sbp": 130,
    "dbp": 85,
    "rr": 18,
    "bt": 36.8,
    "chief_complaint": "Silny ból brzucha.",
    "priority_ktas": 3,
    "is_ai_predicted": true,
    "status_przyjecia": "ZAKONCZONE",
    "patient": {
      "id": 1,
      "pesel": "95061612345",
      "first_name": "Adam",
      "last_name": "Nowak",
      "date_of_birth": "1995-06-16T00:00:00Z",
      "gender": "M",

```

```

    "address": "ul. Dworcowa 12/4, 40-001 Katowice",
    "phone": "+48500600700",
    "email": "adam.nowak@gmail.com",
    "emergency_contact_name": "Anna Nowak (Żona)",
    "emergency_contact_phone": "+48600700800",
    "blood_group": "0+",
    "allergies": "Penicylina",
    "chronic_diseases": "Nadciśnienie tętnicze",
    "created_at": "2026-06-16T15:30:00Z"
  }
}
]

```

4.3.4.6. Pobranie szczegółów pojedynczego przyjęcia

Ścieżka: GET /api/admissions/:id

Opis: Zwraca komplet danych medycznych, pełny wywiad środowiskowy Triage, parametry życiowe oraz automatycznie dołączony pełny obiekt profilu pacjenta. Służy do renderowania szczegółowych widoków w panelu gabinetowym lekarza.

Przykładowy adres URL: http://localhost:8080/api/admissions/1

Schemat odpowiedzi (Response - 200 OK):

```

{
  "id": 1,
  "id_pacjenta": 1,
  "id_osoby_przyjmujacej": 2,
  "id_lekarza_prowadzacego": null,
  "data_przyjecia": "2026-06-18T16:30:58.350609Z",
  "forma_przybycia": "Karetka publiczna",
  "injury": true,
  "mental_status": "Nieprzytomny/Brak reakcji",
  "pain": true,
  "pain_lvl": 9,
  "hr": 145,
  "sbp": 85,
  "dbp": 50,
  "rr": 24,
  "bt": 36.2,
  "chief_complaint": "Pacjent po wypadku samochodowym, podejrzenie urazu wielonarządowego, utrata przytomności.",
  "priority_ktas": 1,
  "is_ai_predicted": true,
  "status_przyjecia": "W_GABINECIE",
  "patient": {
    "id": 1,
    "pesel": "95061612345",
    "first_name": "Adam",
    "last_name": "Nowak",
    "date_of_birth": "1995-06-16T00:00:00Z",
    "gender": "M",
    "address": "ul. Dworcowa 12/4, 40-001 Katowice",
    "phone": "+48500600700",

```



```

    "email": "adam.nowak@gmail.com",
    "blood_group": "0+",
    "allergies": "Penicylina, orzechy arachidowe",
    "chronic_diseases": "Nadciśnienie tętnicze",
    "created_at": "2026-06-16T15:30:00Z"
  }
}

```

4.3.4.7. Pobieranie historii wszystkich wizyt pacjenta na podstawie numeru PESEL

Ścieżka: GET /api/patients/:pesel/admissions

Opis: Przeszukuje relacyjną bazę danych PostgreSQL i wyciąga chronologiczną listę wszystkich historycznych kart przyjęć z dołączonymi danymi pacjenta przypisanymi do konkretnego numeru PESEL. Pozwala lekarzowi prowadzącemu na natychmiastową analizę retrospektywną pacjenta podczas bieżącej wizyty.

Przykładowy adres URL: http://localhost:8080/api/patients/95061612345/admissions

Schemat odpowiedzi (Response - 200 OK):

```

[
  {
    "id": 12,
    "id_pacjenta": 1,
    "data_przyjecia": "2026-06-23T19:40:00Z",
    "chief_complaint": "Silny ból brzucha, tkliwość w prawym podżebrzu.",
    "priority_ktas": 3,
    "status_przyjecia": "ZAKONCZONE",
    "patient": {
      "id": 1,
      "pesel": "95061612345",
      "first_name": "Adam",
      "last_name": "Nowak"
    }
  }
]

```

4.3.4.8. Archiwum i historia wszystkich przyjęć na SOR

Ścieżka: GET /api/admissions/history

Opis: Pobiera pełną, historyczną listę wszystkich zgłoszeń i przyjęć zarejestrowanych w systemie E-SOR. W celu optymalizacji wydajności aplikacji klienckiej i uniknięcia wielokrotnego odpytywania bazy (problem $N + 1$), punkt końcowy automatycznie agreguje dane wejściowe – zwraca unikalny identyfikator przyjęcia wraz z pełnym, zagnieżdżonym obiektem danych osobowo-medycznych przypisanego pacjenta pod kluczem "pacjent". Wyniki domyślnie sortowane są chronologicznie od najnowszych.

Przykładowy adres URL: http://localhost:8080/api/admissions/history

Schemat odpowiedzi (Response - 200 OK):

```

[
  {

```

```

    "id_przyjecia": 12,
    "priority_ktas": 3,
    "status_przyjecia": "ZAKONCZONE",
    "data_przyjecia": "2026-06-23T19:40:00Z",
    "pacjent": {
      "id": 1,
      "pesel": "95061612345",
      "first_name": "Adam",
      "last_name": "Nowak",
      "date_of_birth": "1995-06-16T00:00:00Z",
      "gender": "M",
      "address": "ul. Dworcowa 12/4, 40-001 Katowice",
      "phone": "+48500600700",
      "email": "adam.nowak@gmail.com",
      "emergency_contact_name": "Anna Nowak (Żona)",
      "emergency_contact_phone": "+48600700800",
      "blood_group": "0+",
      "allergies": "Penicylina, orzechy arachidowe",
      "chronic_diseases": "Nadciśnienie tętnicze",
      "created_at": "2026-06-16T15:30:00Z"
    }
  }
}
]

```

4.3.4.9. Anulowanie karty przyjęcia

Ścieżka: DELETE /api/admissions/:id

Opis: Trwale usuwa kartę przyjęcia z systemu i kolejki SOR (np. w przypadku rezygnacji pacjenta lub błędu personelu).

4.3.5. Panel Lekarza i Diagnostyka

Ekran klientki dedykowane dla personelu lekarskiego (Dashboard Lekarza, Ekran Badania, Zlecenia) komunikują się z backendem przy użyciu tokenu JWT z zakodowaną rolą LEKARZ lub ADMIN. W celu zapewnienia maksymalnego bezpieczeństwa, punkty końcowe pobierania danych oraz tworzenia dokumentacji medycznej nie przyjmują identyfikatora lekarza w adresie URL — system automatycznie i bezpiecznie wyodrębnia userID bezpośrednio z kontekstu autoryzacji tokenu sesyjnego.

4.3.5.1. Pobranie zgłoszeń przypisanych do zalogowanego lekarza

Ścieżka: GET /api/doctor/admissions

Opis: Pobiera listę wszystkich pacjentów aktualnie przypisanych do lekarza wykonującego zapytanie, którzy znajdują się w trakcie procesu diagnostyczno-leczniczego (statusy W_GABINECIE lub OCZEKUJE_NA_WYNIKI). Wyniki automatycznie dołączają pełną strukturę danych pacjenta pod kluczem "patient".

Schemat odpowiedzi (Response - 200 OK):

```

[
  {
    "id": 4,
    "id_pacjenta": 4,
    "id_lekarza_prowadzacego": 2,

```

```

    "forma_przybycia": "Pieszo",
    "priority_ktas": 3,
    "status_przyjecia": "W_GABINECIE",
    "data_przyjecia": "2026-06-19T08:24:54Z",
    "patient": {
      "id": 4,
      "pesel": "88112298765",
      "first_name": "Jan",
      "last_name": "Kowalski"
    }
  }
]

```

4.3.5.2. Pobranie historii zgłoszeń lekarza (Zamknięte karty)

Ścieżka: GET /api/doctor/admissions?history=true

Opis: Pobiera historyczne, sfinalizowane zgłoszenia medyczne przypisane do zalogowanego lekarza, w których proces obsługi na SOR został pomyślnie zakończony (status ZAKONCZONE). Zwraca obiekt przyjęcia wraz z kompletem danych pacjenta.

Schemat odpowiedzi (Response - 200 OK):

```

[
  {
    "id": 1,
    "id_pacjenta": 1,
    "id_lekarza_prowadzacego": 2,
    "status_przyjecia": "ZAKONCZONE",
    "data_przyjecia": "2026-06-18T10:45:00Z",
    "patient": {
      "id": 1,
      "pesel": "95061612345",
      "first_name": "Adam",
      "last_name": "Nowak"
    }
  }
]

```

4.3.5.3. Przejęcie pacjenta z poczekalni (Wezwanie do gabinetu)

Ścieżka: PUT /api/admissions/:id/assign

Opis: Lekarz pobiera wybranego pacjenta z głównej kolejki Triage. System przypisuje identyfikator lekarza do karty przyjęcia oraz automatycznie zmienia status pacjenta na W_GABINECIE. Operacja ta wyzwała asynchroniczny broadcast przez WebSocket, usuwając pacjenta z ogólnego monitora poczekalni.

Schemat odpowiedzi (Response - 200 OK):

```

{
  "message": "Pacjent został pomyślnie przyjęty do gabinetu",
  "status": "W_GABINECIE"
}

```

4.3.5.4. Zlecenie badania diagnostycznego

Ścieżka: POST /api/ diagnostic-orders

Opis: Tworzy nowe zlecenie na wykonanie procedury diagnostycznej (np. badania laboratoryjne, obrazowe). W ramach transakcji system automatycznie zmienia status karty przyjęcia na OCZEKUJE_NA_WYNIKI, co sygnalizuje opuszczenie gabinetu na czas realizacji procedury.

Dozwolone wartości pola typ_badania: KREW, RTG, EKG, TK, USG.

Schemat zapytania (Request Body):

```
{
  "id_przyjecia": 4,
  "typ_badania": "KREW",
  "opis_zlecenia": "Morfologia, CRP, elektrolity oraz próby wątrobowe w trybie pilnym."
}
```

Schemat odpowiedzi (Response - 201 Created):

```
{
  "id": 1,
  "message": "Badanie zostało pomyślnie zlecone",
  "status": "OCZEKUJE_NA_WYNIKI"
}
```

4.3.5.5. Finalizacja wizyty lekarskiej (Konsultacja końcowa)

Ścieżka: POST /api/ consultations

Opis: Zamyka proces obsługi pacjenta na oddziale ratunkowym. Lekarz wprowadza oficjalny wywiad medyczny, stawia diagnozę kodowaną w standardzie ICD-10 oraz określa decyzję wyjściową. Karta przyjęcia otrzymuje status ZAKONCZONE.

Dozwolone wartości pola decyzja_wyjsciowa: WYPIS_DO_DOMU, HOSPITALIZACJA, TRANSFER_NA_ODDZIAL, ZGON.

Schemat zapytania (Request Body):

```
{
  "id_przyjecia": 4,
  "wywiad_lekarski": "Tkliwość w prawym podżebrzu. Wyniki badań w normie. Objawy ustąpiły po podaniu leków rozkurczowych.",
  "rozpoznanie_icd10": "K30 - Dyspepsja",
  "decyzja_wyjsciowa": "WYPIS_DO_DOMU"
}
```

Schemat odpowiedzi (Response - 201 Created):

```
{
  "id": 1,
  "message": "Konsultacja zapisana, karta pacjenta została pomyślnie zamknięta"
}
```

4.3.6. Moduł Personelu oraz Publiczny Przegląd Kolejki (RODO)

4.3.6.1. Wyszukiwarka i Katalog Personelu

Ścieżka: GET /api/staff

Opis: Pobiera dynamicznie przefiltrowaną listę zarejestrowanych pracowników medycznych i administracyjnych. Ze względów bezpieczeństwa system filtruje dane wyjściowe i bezwzględnie wycina hasła użytkowników z obiektów odpowiedzi.

Parametry zapytania (Query Params):

- q (opcjonalny): Fraza wyszukiwana asynchronicznie po imieniu lub nazwisku (wyszukiwanie typu *case-insensitive*).
- role (opcjonalny): Dokładny filtr roli systemowej (np. LEKARZ, RATOWNIK, ADMIN).

Schemat odpowiedzi (Response - 200 OK dla /api/staff?role=LEKARZ&q=jan):

```
[
  {
    "ID": 1,
    "FirstName": "Daniel",
    "LastName": "Kowalski",
    "AcademicTitle": "",
    "Role": "LEKARZ",
    "LoginEmail": "jan.kowalski@esor.pl",
    "PasswordHash": "",
    "CreatedAt": "2026-06-16T15:19:28.343935Z"
  }
]
```

4.3.6.2. Anonimowy Strumień Telewizora Poczekalni

Ścieżka: GET (Handshake HTTP) -> ws://localhost:8080/api/ws/admissions/public-queue

Opis: Publiczny, beztokenowy punkt końcowy WebSocket dedykowany dla zbiorczych monitorów i telewizorów powieszonych w fizycznej poczekalni SOR-u. Aby spełnić surowe unijne wymagania prawne w zakresie przetwarzania danych osobowych (RODO), strumień ten przesyła wyłącznie numeryczne identyfikatory biletów oraz odpowiadający im stopień pilności KTAS. Front-end na podstawie kodu KTAS koloruje bilet na odpowiedni kolor (np. 1 - czerwony, 3 - żółty), informując pacjentów o ich pozycji w kolejce bez ujawniania tożsamości czy danych o dolegliwościach.

Schemat przesyłanej wiadomości (Strumień JSON / Broadcast):

```
[
  {
    "numer_biletu": 4,
    "priority_ktas": 3
  },
  {
    "numer_biletu": 7,
    "priority_ktas": 5
  }
]
```

4.4. Środowisko Wytwórcze i Narzędzia Deweloperskie

Implementacja warstwy serwerowej systemu E-SOR została zrealizowana w oparciu o filozofię maksymalnej kontroli nad kodem źródłowym oraz konteneryzację środowiska uruchomieniowego. Wszystkie komponenty backendowe zostały napisane ręcznie bez użycia automatycznych generatorów szkieletów projektowych, co pozwoliło na precyzyjne zarządzanie strukturą aplikacji oraz optymalizację potoków przetwarzania danych.

4.4.1. Zintegrowane Środowisko Programistyczne (IDE)

Głównym narzędziem wykorzystanym do wytworzenia kodu źródłowego był edytor **Visual Studio Code (VS Code)**. Prace programistyczne były prowadzone w 100% manualnie. W celu zapewnienia wysokiej asysty podczas kodowania, środowisko zostało rozbudowane o oficjalne pakiety rozszerzeń dla języka Go (`golang.go`), narzędzia do statycznej analizy kodu (*linters*) oraz automatyczne formatowanie kodu zgodnie ze ścisłym standardem `gofmt`. Ręczna konfiguracja projektu pozwoliła na głębokie zrozumienie cyklu życia żądań HTTP oraz mechaniki działania struktur języka Go.

4.4.2. Lokalna Konteneryzacja i Wirtualizacja Środowiska

W celu odizolowania bazy danych od systemu operacyjnego hosta oraz zapewnienia powtarzalności środowiska deweloperskiego, wykorzystano technologię konteneryzacji:

- **Docker Desktop (Docker Client):** Wykorzystany jako lokalny system zarządzania kontenerami, umożliwiający graficzne monitorowanie zasobów (pamięć RAM, użycie procesora) oraz szybki wgląd w strumień logów systemowych bazy danych PostgreSQL i serwera Go w czasie rzeczywistym.
- **Docker Compose:** Użyty do deklaratywnego definiowania i orkiestracji wielokontenerowej architektury. Plik konfiguracyjny automatycznie zarządzał sieciami wewnętrznymi (*networks*) oraz wolumenami pamięci masowej (*volumes*), co zagwarantowało trwałość danych testowych pacjentów i personelu medycznego pomiędzy restartami platformy.

4.4.3. Testowanie i Walidacja Punktów Końcowych HTTP (REST)

Do asynchronicznego testowania operacji CRUD oraz symulacji pełnego przepływu medycznego (od logowania personelu, przez rejestrację i procedurę Triage) wykorzystano wbudowane w środowisko edytora rozszerzenie **Thunder Client** (wersja darmowa).

Narzędzie to posłużyło jako lokalny klient HTTP, zastępując cięższe programy zewnętrzne. Umożliwiło ono precyzyjne konstruowanie żądań typu POST, GET, PUT, PATCH i DELETE, ręczne zarządzanie nagłówkami autoryzacyjnymi (*Authorization: Bearer <token>*) oraz weryfikację kodów odpowiedzi HTTP (m.in. 200 OK, 201 Created, 400 Bad Request, 500 Internal Server Error) wraz z walidacją struktur JSON zwracanych przez silnik Gin Gonic.

4.4.4. Testowanie Dwukierunkowych Strumieni WebSocket

Weryfikacja reaktywności systemu w czasie rzeczywistym oraz poprawnego działania mechanizmu rozgłaszania kolejki (*broadcast*) wymagała przetestowania asynchronicznych protokołów `ws://`. Proces ten zrealizowano przy użyciu przeglądarki internetowej **Brave**, wykorzystując jej wbudowaną, natywną konsolę deweloperską (*Brave DevTools -> Console*).

Test polegał na uruchomieniu niskopoziomowego skryptu klienckiego w języku JavaScript, który symulował zachowanie dedykowanego monitora zbiorczego w poczekalni lub aplikacji mobilnej. Skrypt ten pomyślnie realizował proces uścisku dłoni (Handshake) z uwzględnieniem hybrydowej autoryzacji tokenem JWT zaszytym w parametrze URL, a następnie asynchronicznie

nasłuchiwał ramek sieciowych i logował spływające pakiety danych JSON po każdej mutacji stanu bazy.

Skrypt testowy JavaScript użyty w konsoli Brave DevTools:

```
// TUTAJ WKLEJ SWÓJ SKRYPT JAVASCRIPT DO TESTOWANIA WEBSOCKETÓW
// Przykład:
// const ws = new WebSocket('ws://localhost:8080/api/ws/admissions/queue?token=...');
// ws.onmessage = (event) => console.log('Otrzymano kolejkę:', JSON.parse(event.data));
```

5. Moduł Analityczny Sztucznej Inteligencji

5.1. Architektura mikrousługi Python / FastAPI

5.1.1. Opis Architektury

Architektura mikrousługi modułu sztucznej inteligencji została zorientowana wokół asynchronicznego frameworku **FastAPI** oraz w pełni zintegrowana z modelem uczenia maszynowego w postaci **własnego drzewa decyzyjnego**. Rdzeniem wydajnościowym tej architektury jest mechanizm bezpiecznego, jednorazowego ładowania struktury drzewa z serializowanego pliku binarnego **drzewo_model.pickle** bezpośrednio do pamięci RAM przy starcie kontenera, co eliminuje kosztowny narzut wejścia/wyjścia podczas obsługi pytań przychodzących z backendu. Ze względu na uruchamianie kodu w izolowanym środowisku Docker przy użyciu serwera **Uvicorn**, w architekturze zastosowano niskopoziomowy mechanizm manipulacji przestrzenią nazw modułów (`sys.modules`), który dynamicznie mapuje referencje klas `DecisionNode` oraz głównego klasyfikatora `DecisionTreeClassifierRaw` na dedykowany moduł `tree_classes`, gwarantując bezbłędną deserializację obiektów przez bibliotekę **pickle**.

5.1.2. Opis wybranych narzędzi i bibliotek

- **FastAPI** - nowoczesny, szybki i asynchroniczny framework sieciowy dla języka Python. Służy do budowania interfejsów API, a w projekcie odpowiada za wystawienie endpointów, obsługę żądań sieciowych z backendu oraz automatyczną walidację przesyłanych danych.
- **Uvicorn** - produkcyjny serwer sieciowy typu ASGI (Asynchronous Server Gateway Interface) dla Pythona. Działa jako bezpośredni silnik uruchomieniowy wewnątrz kontenera, który fizycznie nasłuchuje na porcie sieciowym i przekazuje ruch do aplikacji FastAPI.
- **Pickle** - wbudowana biblioteka języka Python służąca do serializacji i deserializacji obiektów. W systemie odpowiada za odczyt i odtworzenie zapisanego wcześniej na dysku modelu drzewa decyzyjnego.
- **Sys** - standardowy, niskopoziomowy moduł systemowy Pythona. Służy do zarządzania środowiskiem uruchomieniowym, a w tym konkretnym przypadku do manipulacji słownikiem załadowanych modułów, co pozwala na poprawne odtworzenie struktury klas modelu w specyficznym środowisku serwera Uvicorn.

5.2. Opis matematyczny i teoretyczny wybranego modelu uczenia maszynowego

Zaimplementowany moduł klasyfikacji pacjentów na Szpitalnym Oddziale Ratunkowym (SOR) opiera się na algorytmie **drzewa decyzyjnego** (ang. *Decision Tree*), zaimplementowanym w postaci czystej klasy strukturalnej `DecisionTreeClassifierRaw`. Model ten jest nieparametrycznym algorytmem nadzorowanego uczenia maszynowego, który dokonuje podziału przestrzeni cech wejściowych na rozłączne hiperobszary przy użyciu hierarchicznych reguł warunkowych.

5.2.1. Struktura Topologiczna Drzewa

Algorytm operuje na grafie acyklicznym reprezentowanym przez klasę `DecisionNode`. Każdy węzeł decyzyjny przechowuje informację o indeksie cechy podziału f , wartości progowej t oraz referencje do dwóch poddrzew potomnych: lewego N_L (dla danych spełniających warunek $x_f \leq t$) oraz prawego N_R (dla danych spełniających warunek $x_f > t$). Węzły końcowe, czyli liście, przechowują finalną wartość predykcji klasy priorytetu v , wyznaczaną za pomocą metody większościowej:

$$v = \arg \max_{c \in C} \sum_{i=1}^N I(y_i = c)$$

Gdzie C oznacza zbiór możliwych klas priorytetu triage, N to liczba próbek w danym liściu, a I jest funkcją wskaźnikową.

5.2.2. Metryka Nieczystości Węzła (Indeks Giniego)

Kluczowym elementem matematycznym algorytmu jest kryterium optymalizacji podziału przestrzeni danych. W zaimplementowanej metodzie `_gini` stopień nieczystości (heterogeniczności) zbioru danych D w danym węźle mierzony jest za pomocą **Indeksu Giniego** (ang. *Gini Impurity*). Matematyczna definicja wskaźnika dla zbioru prób o strukturze klasowej C przyjmuje postać:

$$I_G(D) = 1 - \sum_{c \in C} p_c^2$$

Gdzie p_c reprezentuje prawdopodobieństwo empiryczne, czyli częstość wystąpienia klasy c w podzbiorze D , obliczane w kodzie jako:

$$p_c = \frac{|\{i \in D : y_i = c\}|}{|D|}$$

5.2.3. Optymalizacja Podziału i Zysk Informacyjny

W celu wyznaczenia optymalnego podziału przestrzeni w metodzie `_best_split`, algorytm dokonuje iteracyjnego przeglądu wszystkich dostępnych cech wejściowych oraz wszystkich unikalnych wartości progowych t występujących w danych treningowych. Dla każdego potencjalnego podziału zbiór D jest dzielony na dwa podzbiory: D_L oraz D_R . Jakościowa ocena podziału mierzona jest za pomocą maksymalizacji zysku informacyjnego (funkcja `gain`), zdefiniowanego jako różnica między nieczystością węzła macierzystego a ważoną sumą nieczystości węzłów potomnych:

$$\text{Gain}(D, f, t) = I_G(D) - \left(\frac{|D_L|}{|D|} I_G(D_L) + \frac{|D_R|}{|D|} I_G(D_R) \right)$$

Algorytm wybiera parę (f^*, t^*) , która maksymalizuje powyższą funkcję zysku:

$$(f^*, t^*) = \arg \max_{f, t} \text{Gain}(D, f, t)$$

5.2.4. Kryteria Stopu i Rekurencja

Proces budowy drzewa (`_build_tree`) przebiega rekurencyjnie w głąb (ang. *top-down induction*). Aby zapobiec zjawisku przeuczenia (ang. *overfitting*), wdrożono trzy niezależne kryteria stopu, których spełnienie wymusza natychmiastowe utworzenie węzła-liścia:

1. Osiągnięcie maksymalnej dopuszczalnej głębokości drzewa ($\text{depth} \geq \text{max_depth}$).

2. Spadek liczby próbek w węźle poniżej progu podziału ($|D| < \text{min_samples_split}$).
3. Całkowita jednorodność klasowa podzbioru ($|C| = 1$).

5.2.5. Proces Predykcji

Faza inferencji (metody `predict` oraz `_predict_row`) realizowana jest poprzez deterministyczne przejście od węzła głównego (korzenia) w dół struktury grafu. Dla pojedynczego wektora cech pacjenta x algorytm w każdym napotkanym węźle wewnętrznym sprawdza warunek skrajny:

$$\begin{aligned} x_f \leq t &\text{ implies przejdź do lewego poddrzewa } N_L \\ x_f > t &\text{ implies przejdź do prawego poddrzewa } N_R \end{aligned}$$

Proces ten powtarza się rekurencyjnie aż do osiągnięcia węzła, dla którego funkcja `node.is_leaf()` zwraca wartość prawdziwą, co skutkuje zwróceniem przypisanej klasy priorytetu medycznego.

5.3. Proces inżynierii cech (Feature Engineering) i wektor wejściowy parametrów życiowych

Proces przygotowania danych w module SOR AI opiera się na koncepcji selekcji cech o najwyższej wartości predykcyjnej w kontekście medycyny ratunkowej. Zamiast operować na pełnej historii choroby pacjenta, dokonano redukcji wymiarowości danych do zestawu pięciu kluczowych, mierzalnych parametrów fizjologicznych.

W ramach inżynierii cech zastosowano rygorystyczną walidację dziedzinową na poziomie interfejsu API, co stanowi warstwę czyszczenia danych przed ich przekazaniem do modelu. Działanie to odcina szum informacyjny oraz potencjalne błędy personelu lub aparatury medycznej (np. ujemne wartości tętna). Finalnym produktem tego procesu jest transformacja surowego obiektu JSON do postaci zunifikowanego, pięciowymiarowego wektora numerycznego `Fratures`, który stanowi bezpośredni wsad wejściowy dla algorytmu inferencji:

$$\text{fratures} = [\text{patient_data}_{\text{age}}, \text{patient_data}_{\text{hr}}, \text{patient_data}_{\text{sbp}}, \text{patient_data}_{\text{dbp}}, \text{patient_data}_{\text{pain_lvl}}] \in \mathbb{N}^5$$

Każdy element wektora reprezentuje określoną cechę o ściśle zdefiniowanym znaczeniu:

- `patient_dataage` – wiek pacjenta w latach ($0 \leq \text{patient_data}_{\text{age}} \leq 120$), pozwalający modelowi uwzględnić odmienność norm fizjologicznych u dzieci i osób starszych.
- `patient_datahr` – tętno wyrażone w uderzeniach na minutę ($20 \leq \text{patient_data}_{\text{hr}} \leq 250$), będące kluczowym indykátorem wydolności krążeniowej.
- `patient_datasbp` – ciśnienie skurczowe krwi ($40 \leq \text{patient_data}_{\text{sbp}} \leq 250$).
- `patient_datadbp` – ciśnienie rozkurczowe krwi ($40 \leq \text{patient_data}_{\text{dbp}} \leq 250$).
- `patient_datapain_lvl` – intensywność bólu zadeklarowana przez pielęgniarkę ($0 \leq \text{patient_data}_{\text{pain_lvl}} \leq 10$).

Utrzymanie stałej struktury oraz niezmienniej kolejności elementów w wektorze `features` jest krytycznym warunkiem determinizmu matematycznego drzewa decyzyjnego, ponieważ algorytm w każdym węźle odwołuje się bezpośrednio do konkretnego indeksu tej tablicy.

5.4. Proces trenowania modelu, dobór hiperparametrów i zestaw danych uczących

Proces budowy i optymalizacji modelu predykcyjnego dla modułu AI zrealizowano w oparciu o zestaw danych treningowych oraz procedurę przeszukiwania siatki hiperparametrów. Wykorzystany w projekcie zbiór danych nosi nazwę **Emergency Service - Triage Application**

i został pozyskany z platformy naukowej Kaggle. Przed przystąpieniem do fazy właściwego uczenia maszynowego, baza danych została poddana procesowi czyszczenia, polegającemu na bezwzględny usunięciu rekordów uszkodzonych lub zawierających anomalie fizjologiczne. W przypadku rekordu z pustą cechą, wypełniano ją medianą dla danego parametru. W ramach selekcji zmiennych, spośród wszystkich dostępnych w bazie parametrów, wyizolowano pięć kluczowych cech wejściowych o najwyższej wartości diagnostycznej: wiek pacjenta (`age`), tętno (`hr`), ciśnienie skurczowe (`sbp`), ciśnienie rozkurczowe (`dbp`) oraz subiektywną skalę bólu (`pain_lvl`). Jako zmienną celu (wartość szukaną) zdefiniowano stopień segregacji medycznej zapisany w kolumnie `priority_level`. Tak przygotowany zbiór został losowo podzielony na podzbiór treningowy (80%) oraz podzbiór testowy (20%) przy zachowaniu spójności par cecha-etykieta.

W celu maksymalizacji zdolności generalizacji autorskiego klasyfikatora `DecisionTreeClassifierRaw` oraz ochrony przed zjawiskiem przeuczenia, przeprowadzono systematyczne testy optymalizacyjne w dwuwymiarowej przestrzeni hiperparametrów. Badaniu poddano kombinacje maksymalnej głębokości drzewa oraz minimalnej liczby próbek wymaganej do podziału węzła wewnętrznego, definiując dyskretne przestrzenie poszukiwań przy użyciu stałego kroku matematycznego:

$$\begin{aligned}\max_depth &\in \{3, 5, 7, 9, \dots, 21\} \\ \min_samples_split &\in \{3, 5, 7, 9, \dots, 21\}\end{aligned}$$

Algorytm indukcji drzewa decyzyjnego został uruchomiony iteracyjnie dla każdego możliwego wariantu par parametrów ze zdefiniowanej siatki, a jakość każdego podziału mierzona była zyskiem informacyjnym na podstawie indeksu nieczystości Giniego. Na podstawie rygorystycznej analizy porównawczej wyników klasyfikacji na niezależnym zbiorze walidacyjnym, jako konfigurację optymalną wyznaczono wartości: `max_depth = 13` oraz `min_samples_split = 3`. Wybrana kombinacja hiperparametrów pozwoliła modelowi na zbudowanie odpowiednio głębokiej i szczegółowej struktury reguł decyzyjnych zdolnej do wychwytywania nieliniowych zależności medycznych, chroniąc jednocześnie strukturę grafu przed nadmiernym rozbudowaniem na poziomach bliskich liściom. Ostatecznie wytrenowany model został zserializowany do formatu binarnego `drzewo_model.pickle` w celu integracji z produkcyjną warstwą FastAPI.

5.5. Analiza metryk wydajnościowych modelu (Accuracy, Precision, Recall, F1-Score)

Ocena jakościowa i ilościowa wytrenowanego klasyfikatora `DecisionTreeClassifierRaw` została przeprowadzona na wydzielonym zbiorze testowym obejmującym $N = 293$ rekordów (co stanowi 20% z puli 1465 wierszy bazy danych). Ewaluacji dokonano przy optymalnej konfiguracji hiperparametrów (`max_depth = 13`, `min_samples_split = 3`). Do całościowej analizy błędów wykorzystano macierz pomyłek oraz wyznaczone na jej podstawie cztery fundamentalne metryki klasyfikacji wieloklasowej obliczane metodą *One-vs-All*: dokładność ogólną, precyzję, czułość (*Recall*) oraz średnią harmoniczną *F1-Score*.

Globalna dokładność modelu wyniosła 0.52, co oznacza, że 52% wszystkich decyzji segregacji medycznej na zbiorze testowym było w pełni bezbłędnych. Choć w klasycznych systemach IT wskaźnik ten mógłby wydawać się umiarkowany, szczegółowa analiza macierzy pomyłek w ujęciu klinicznym dowodzi, że model wykazuje wyjątkowo pożądaną charakterystykę dla systemów wspomagania decyzji na Szpitalnych Oddziałach Ratunkowych.

Klasa (Priority)	Precyzja	Czułość (Recall)	F1-Score
1 (Stan krytyczny)	0.84	1.00	0.91
2 (Stan bardzo pilny)	0.37	0.22	0.28
3 (Stan pilny)	0.45	0.54	0.49
4 (Stan stabilny)	0.47	0.45	0.46
5 (Stan lekki)	0.70	0.63	0.66

Tabela 1: Wyniki pomiarów skuteczności modelu AI.

Kluczowym osiągnięciem algorytmu jest uzyskanie perfekcyjnej czułości na poziomie $R_1 = 1.00$ (100%) dla priorytetu równego 1. Oznacza to, że model poprawnie zidentyfikował wszystkich pacjentów w stanie krytycznym i nie przeoczył ani jednego pacjenta w stanie krytycznym. Towarzysząca temu wysoka precyzja $P_1 = 0.84$ (84%) potwierdza minimalny odsetek fałszywych alarmów (tylko kilka przypadków błędnego zakwalifikowania pacjentów z niższych klas do priorytetu pierwszego). Przekłada się to na bardzo wysoki wskaźnik $F1_1 = 0.91$, co jest wynikiem doskonałym w medycynie ratunkowej, gdzie nadrzędnym celem jest bezwzględne bezpieczeństwo pacjenta.

Największą trudność klasyfikacyjną model napotyka na granicy klas pośrednich (Priority 2 oraz Priority 3), gdzie czułość dla klasy drugiej spada do poziomu 0.22. Analiza macierzy pomyłek wskazuje, że aż 21 pacjentów o rzeczywistym priorytecie 2 zostało sklasyfikowanych jako priorytet 3. Wynika to z faktu, że parametry życiowe (takie jak tętno czy ciśnienie krwi) w stanach „bardzo pilnych” i „pilnych” często nakładają się na siebie w sensie matematycznym, tworząc silnie nieliniowe i rozmyte granice decyzyjne w przestrzeni cech. Z kolei dla pacjentów o najniższym statusie ratunkowym (Priority 5), model ponownie osiąga stabilne i wysokie wyniki ($P_5 = 0.70$, $R_5 = 0.63$, $F1_5 = 0.66$), co pozwala na skuteczne odsianie przypadków lekkich od realnych zagrożeń. Podsumowując, mimo umiarkowanej ogólnej celności, model wykazuje asymetrię błędów bezpieczną dla człowieka – myli się głównie o jedną klasę w obszarach bezpiecznych, bezbłędnie izolując najpoważniejsze przypadki.

Klasa (Rzecz. / Przew.)	1	2	3	4	5
Priority 1	32	0	0	0	0
Priority 2	4	11	21	11	2
Priority 3	1	10	51	30	2
Priority 4	0	9	35	40	4
Priority 5	1	0	6	4	19

Tabela 2: Macierz pomyłek

5.6. Integracja i komunikacja synchroniczna Backend-AI

Warstwa integracyjna systemu SOR opiera się na synchronicznym modelu komunikacji sieciowej, realizowanym w architekturze klient-serwer za pomocą protokołu HTTP/1.1 oraz formatu wymiany danych JSON. W tym ujęciu główny backend aplikacji pełni rolę klienta inicjującego zapytanie, natomiast mikrousługa FastAPI działa jako bezstanowy serwer obliczeniowy. Proces integracji rozpoczyna się w momencie przesłania parametrów życiowych pacjenta na dedykowany, produkcyjny punkt końcowy typu POST o ścieżce `/api/triage`. Komunikacja odbywa się w sposób synchroniczny i blokujący dla danego wątku sieciowego, co oznacza, że backend

oczekuje na odpowiedź systemu AI, utrzymując otwarte połączenie TCP. Odebrane przez serwer ASGI dane wejściowe są natychmiastowo mapowane na strukturę walidacyjną `TriageRequest`, która za pomocą biblioteki Pydantic weryfikuje poprawność typów oraz dopuszczalne zakresy fizjologiczne przed dopuszczeniem danych do warstwy predykcyjnej.

Bezpieczeństwo i stabilność architektury integracyjnej są gwarantowane przez rozbudowany mechanizm obsługi błędów i odporności na awarie. W przypadku wystąpienia błędu krytycznego podczas inicjalizacji kontenera, na przykład w wyniku braku pliku binarnego `drzewo_model.pickle` na dysku serwera, zmienna `modelAI` przyjmuje wartość `None`. Próba wywołania predykcji w takim stanie skutkuje natychmiastowym przerwaniem operacji i zwróceniem do głównego backendu semantycznego kodu błędu HTTP 503 (*Service Unavailable*) wraz z komunikatem o niedostępności modelu. Jeżeli warunki startowe zostały spełnione, ustrukturyzowany wektor cech trafia do metody `.predict()` drzewa decyzyjnego, a uzyskana odpowiedź jest jawnie konwertowana na typ całkowitoliczbowy (`int`). Finalny wynik jest pakowany w asynchroniczną strukturę zwrotną `TriageResponse` jako pole `priority_level` i odsyłany z kodem statusu HTTP 200 OK. Ewentualne nieprzewidziane anomalie obliczeniowe wewnątrz silnika AI są izolowane globalnym blokiem `try-except`, który przechwytuje wyjątki i mapuje je na błąd HTTP 500 (*Internal Server Error*), co zapobiega awaryjnemu zamknięciu procesu FastAPI i pozwala głównemu backendowi na prawidłową, kontrolowaną reakcję systemową.

5.7. Komunikacja z modelem poprzez API

Bezpośrednia interakcja z warstwą analityczną mikrouслуги została ustrukturyzowana przy użyciu standardu architektonicznego REST API, realizowanego przez asynchroniczne punkty końcowe wystawione przez framework FastAPI. Podstawowym mechanizmem weryfikacji statusu operacyjnego serwera jest bezstanowy punkt końcowy obsługiwany metodą GET pod adresem głównym (`/`). Przeznaczeniem tego endpointu jest szybka diagnostyka łączności sieciowej typu *Ping*. Wytlumaczeniem poprawnego działania usługi w tym punkcie jest natychmiastowe zwrócenie prostego obiektu JSON o strukturze słownikowej: `{"message": "Hellooooo SOR"}` z kodem statusu HTTP 200 OK, co potwierdza gotowość serwera ASGI do przetwarzania dalszych, bardziej złożonych zapytań.

Właściwy proces wnioskowania i komunikacji z modelem AI zachodzi w sposób synchroniczny za pośrednictwem dedykowanego punktu końcowego o ścieżce `/api/triage`, który akceptuje wyłącznie żądania przesyłane metodą POST. Taki dobór metody HTTP jest podyktowany koniecznością bezpiecznego przekazywania struktur danych w ciele zapytania. Klient (backend) inicjuje komunikację, wysyłając surowy dokument w formacie JSON, zawierający asymetryczny zestaw pięciu numerycznych parametrów życiowych pacjenta, zmapowanych zgodnie z regułami modelu `TriageRequest`. Przykładowe, poprawne semantycznie żądanie przyjmuje następującą postać strukturalną:

```
{
  "age": 45,
  "hr": 110,
  "sbp": 140,
  "dbp": 90,
  "pain_lvl": 7
}
```

W przypadku pomyślnej weryfikacji, parametry są przekazywane do wewnętrznej metody `.predict()` załadowanego klasyfikatora `DecisionTreeClassifierRaw`. Model przetwarza wektor wej-

ściowy i zwraca jedną, wyliczoną matematycznie klasę priorytetu ratunkowego. Ta informacja jest pakowana jako odpowiedź synchroniczna w ustrukturyzowany obiekt JSON zgodny ze schematem `TriageResponse`. Odpowiedź zwrotna ogranicza narzut sieciowy do absolutnego minimum i zawiera wyłącznie kluczową dla personelu medycznego informację o stopniu pilności:

```
{
  "priority_level": 3
}
```

6. Projekt i Implementacja Aplikacji Klientkiej (Frontend)

6.1. Wybór technologii i frameworka

Do stworzenia graficznego interfejsu użytkownika (GUI) systemu E-SOR wybrano framework **Flutter** opracowany przez Google. Decyzja ta była podyktowana kilkoma kluczowymi czynnikami inżynierskimi:

- **Wieloplatformowość (Cross-platform):** Flutter pozwala na utrzymanie jednej, spójnej bazy kodu (Single Codebase) w języku Dart. Z tego samego źródła generowane są natywne pakiety instalacyjne dla systemów mobilnych (Android, iOS), desktopowych (macOS, Windows) oraz zoptymalizowane aplikacje webowe. Jest to kluczowe dla systemu szpitalnego, gdzie personel korzysta z różnych urządzeń (tablety na salach, monitory na biurkach).
- **Wydaźność renderowania:** Flutter omija tzw. „mosty” (bridges) charakterystyczne dla innych technologii hybrydowych (np. React Native). Zamiast tego wykorzystuje własny, wysokowydajny silnik renderujący (Impeller/Skia) w oparciu o sprzętową akcelerację GPU. Dzięki temu aplikacja działa płynnie z natywną częstotliwością odświeżania (60/120 Hz).
- **Szybkość iteracji (Hot Reload):** Narzędzie to znacząco przyspieszyło proces deweloperski, pozwalając na wprowadzanie zmian w interfejsie graficznym w ułamkach sekund, bez utraty bieżącego stanu aplikacji.

6.2. Architektura aplikacji klientkiej

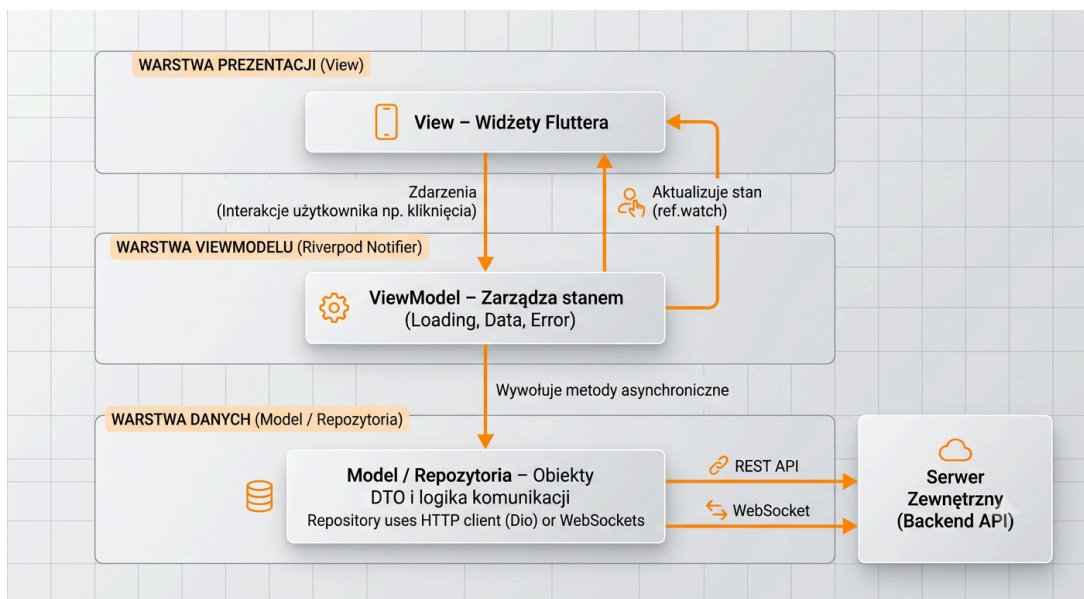
W celu zachowania czystości kodu oraz ułatwienia testowania i skalowalności aplikacji, w warstwie klientkiej rygorystycznie zastosowano wzorzec **Model-View-ViewModel (MVVM)**, zorganizowany według paradygmatu **Feature-First**.

Zamiast tradycyjnego grupowania plików według ich technicznej roli (np. wszystkie modele w jednym folderze, wszystkie widoki w innym), kod został podzielony modułowo, zgodnie z domenami biznesowymi (np. `admissions`, `patients`, `auth`). Każdy z modułów biznesowych wewnętrznie realizuje założenia MVVM:

- **Model (Domain & Data):** Warstwa absolutnie niezależna od interfejsu użytkownika. Obejmuje klasy danych (Encje), obiekty DTO oraz interfejsy repozytoriów. Komunikuje się z zewnętrznym backendem API (odpowiednik źródła danych) i przetwarza surowe informacje. Zwraca rezultaty w postaci algebraicznych typów danych.
- **ViewModel (Presentation):** Pełni rolę inteligentnego bufora. Odbiera surowe dane z Modelu, formatuje je, transformuje i eksponuje jako reaktywny stan gotowy do narysowania. Przechwytuje również zdarzenia z interfejsu (Komendy użytkownika). ViewModel nigdy nie importuje plików związanych z warstwą renderowania (tzw. widoków).
- **View (UI):** Czysto „pasywna” warstwa renderowania oparta na widżetach Fluttera. Jej jedynym zadaniem jest nasłuchiwanie na zmiany stanu udostępniane przez ViewModel

oraz deklaratywne przerysowywanie interfejsu na ekranie. Widok nie posiada własnej logiki decyzyjnej ani nie wykonuje bezpośrednich żądań sieciowych.

Dodatkowo implementacja w dużej mierze czerpie z paradygmatów **programowania funkcyjnego**.



Rysunek 2: Architektura warstwowa MVVM z wykorzystaniem biblioteki Riverpod

6.3. Wykorzystane biblioteki i zarządzanie stanem

System E-SOR opiera się na wyselekcjonowanym zestawie bibliotek zewnętrznych (zależności pub.dev), które stanowią fundament stabilności:

1. Zarządzanie stanem - flutter_riverpod oraz riverpod_annotation:

Riverpod stanowi centralny kręgosłup architektoniczny frontendu. Odpowiada za wstrzykiwanie zależności (Dependency Injection) pomiędzy warstwami, reaktywne odświeżanie interfejsu oraz bezpieczny cykl życia obiektów ViewModel. Wykorzystanie generatorów kodu (riverpod_generator) zminimalizowało ilość powtarzalnego kodu blokowego (boilerplate).

2. Nawigacja deklaratywna - go_router:

Służy do głębokiego zarządzania trasami w aplikacji. W przeciwieństwie do standardowego, stosowego systemu nawigacji (Imperative Routing), go_router umożliwia zdefiniowanie jawnego drzewa adresów URL, co ułatwiło implementację głębokiego linkowania (Deep Linking) w wersji webowej oraz sprzężenie ścieżek z uprawnieniami RBAC.

3. Komunikacja sieciowa i asynchroniczna - dio i web_socket_channel:

Za żądania REST odpowiada klient HTTP dio, który wspiera globalne interceptory nagłówków (automatyczne dodawanie tokenów JWT do każdego zapytania). Z kolei web_socket_channel utrzymuje dwukierunkowe strumienie danych w czasie rzeczywistym z backendem Go.

4. Bezpieczna obsługa wyjątków - fpdart:

Biblioteka wprowadzająca struktury ze świata programowania funkcyjnego. Wykorzystano typ Either<Failure, Success>, który wymusza na programiście obsłużenie zarówno wariantu pomyślnego, jak i potencjalnego błędu podczas kompilacji. Zapewnia to, że aplikacja kliencka nie ulegnie zamknięciu (Crash) w przypadku niespodziewanego błędu serwera.

5. Serializacja danych i struktury niemutowalne - `freezed` i `json_serializable`:

Użyte do generowania bezpiecznych typologicznie obiektów DTO i encji, które są domyślnie niemutowalne (Immutable). Zapewnia to brak skutków ubocznych (Side-effects) podczas przekazywania stanu wewnątrz aplikacji.

6. Bezpieczeństwo danych wrażliwych - `flutter_secure_storage`:

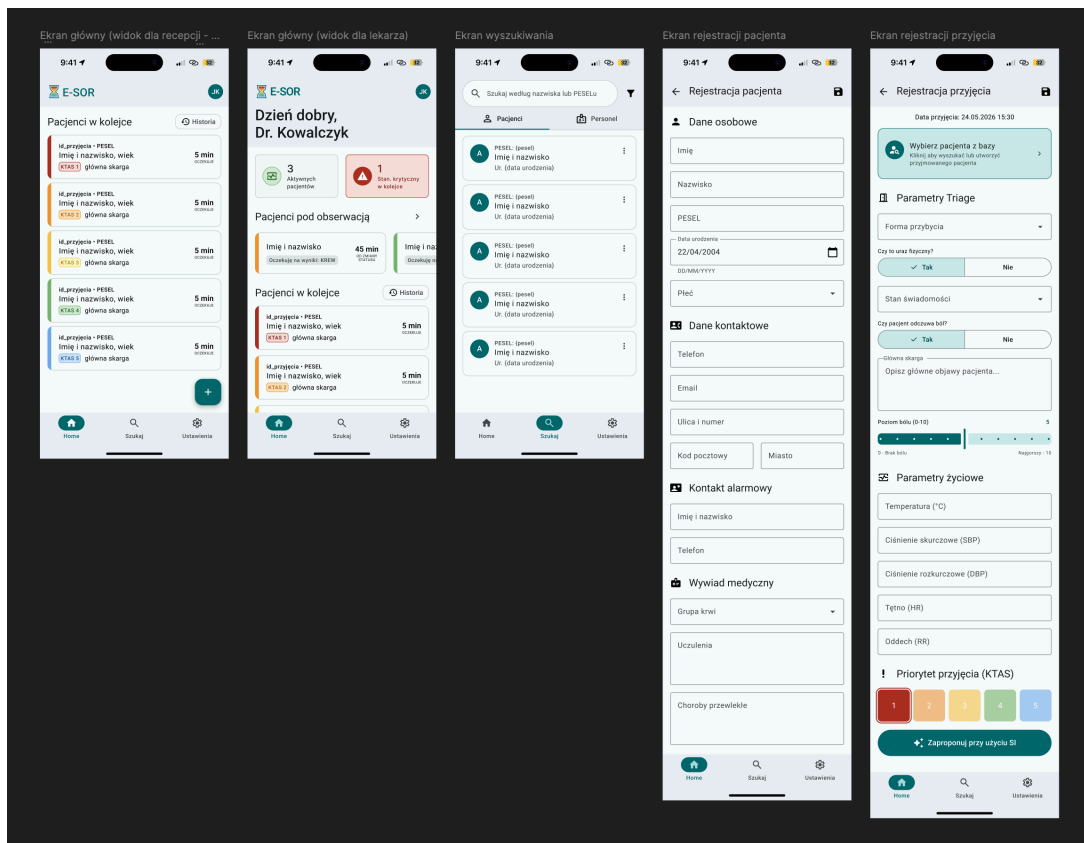
Odpowiada za szyfrowane przechowywanie danych autoryzacyjnych (Tokenów JWT) w specjalnych, chronionych enklawach systemowych (Keychain w systemie iOS/macOS oraz Keystore w systemie Android).

6.4. Faza projektowania (UX/UI)

Z uwagi na specyfikę pracy ratowników medycznych i lekarzy pod dużą presją czasu i w stresujących warunkach szpitalnych, interfejs graficzny musiał zostać zaprojektowany ze szczególnym naciskiem na czytelność i użyteczność.

Przed rozpoczęciem właściwego kodowania aplikacji, główne ekrany zostały rozrysowane i zaprotypowane w narzędziu graficznym **Figma**. Kluczowe założenia użyteczności:

- **Wysoki kontrast i duża czcionka:** Karta pacjenta oraz lista kolejki muszą być czytelne z odległości, bez konieczności wyteżania wzroku.
- **Zminimalizowanie liczby kliknięć:** Krytyczne funkcjonalności, takie jak zmiana priorytetu Triage, zostały wyciągnięte na główny plan, omijając głębokie zagnieżdżenia menu.
- **Wyraźna kategoryzacja kolorystyczna:** Przypisanie pacjentom określonych z góry barw (Czerwony, Żółty, Zielony itd.) na liście dashboardu.
- **Responsywność i skalowalność:** Interfejs powinien dopasowywać się optymalnie zarówno do pionowych ekranów smartfonów, jak i panoramicznych monitorów biurowych gabinetów lekarskich.



Rysunek 3: Projektu interfejsu aplikacji stworzony przy użyciu narzędzia Figma

6.5. Struktura katalogów i plików (Feature-First)

Projekt frontendowy został zorganizowany wokół funkcji biznesowych (Features), co minimalizuje zjawisko rozpraszania kodu (Spaghetti Code) typowe dla aplikacji budowanych metodą tradycyjną (np. dzielenie na foldery `models`, `views`, `controllers` w głównym katalogu).

Poniżej przedstawiono zrzut głównego drzewa katalogów `/lib` reprezentujący ostateczną architekturę systemu:

```
lib/
├── core/
│   ├── constants/
│   ├── network/
│   ├── providers/
│   ├── routing/
│   └── theme/
├── features/
│   ├── admissions/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   ├── auth/
│   ├── dashboard/
│   ├── doctor/
│   ├── patients/
│   ├── public_board/
│   └── search/
│       # Elementy współdzielone (konfiguracja globalna)
│       # Stałe wartości, kolory, wymiary
│       # Klient HTTP (Dio) oraz WebSocketService
│       # Globalni dostawcy stanu (np. SessionProvider)
│       # Konfiguracja ścieżek aplikacji (GoRouter)
│       # Implementacja Material 3 Design
│       # Moduły biznesowe aplikacji (Feature-First)
│       # Rejestracja Przyjęć (Wywiad Triage)
│       # Repozytoria i zapytania API
│       # Modele danych (Encje Freezed, DTO)
│       # Widoki (UI) i modele widoku (ViewModels)
│       # Autoryzacja i logowanie
│       # Główny panel i zarządzanie nawigacją
│       # Konsultacja medyczna i Panel Lekarza
│       # Zarządzanie kartotekami pacjentów
│       # Widok pasywny dla monitorów TV (Poczekalnia)
│       # Wyszukiwarka pacjentów i historia
```



```

|   ├── settings/      # Konfiguracja profilu pracownika
|   ├── staff/         # Baza personelu medycznego
|   └── shared/        # Uniwersalne elementy używane w wielu modułach
|       ├── utils/     # Funkcje pomocnicze, walidatory, formatery
|       └── widgets/   # Globalne kontrolki UI (przyciski, inputy, dialogi)
└── main.dart          # Punkt wejścia aplikacji i konfiguracja Riverpod

```

Dzięki powyższej strukturze, każdy moduł (**feature**) stanowi niezależną domenę. Skutkuje to dużo lepszą separacją logiki biznesowej, a co za tym idzie ułatwia proces wdrażania (onboarding) nowych członków zespołu programistycznego oraz modyfikację konkretnego ekranu bez obaw o uszkodzenie innych komponentów systemu.

6.6. Opis głównych modułów i funkcjonalności systemu

6.6.1. Moduł Autoryzacji (Logowanie i kontrola sesji)

Cel biznesowy: Ograniczenie dostępu do wrażliwych danych medycznych pacjentów wyłącznie dla uwierzytelnionego personelu Szpitalnego Oddziału Ratunkowego. Moduł pozwala na identyfikację roli użytkownika (np. Lekarz, Pielęgniarka) i przypisanie mu odpowiednich uprawnień.

Opis techniczny: Architektura modułu oparta jest na rygorystycznym oddzieleniu widoku od stanu. Formularz jest zasilany przez `AuthViewModel`, który reaguje na akcje użytkownika. Komunikacja z repozytorium uwierzytelniania opiera się o asynchroniczne żądania autoryzacyjne. Po pomyślnym otrzymaniu kryptograficznego tokenu JWT z backendu, jest on automatycznie przechowywany na urządzeniu przy użyciu biblioteki `flutter_secure_storage`. Wstrzyknięcie globalnego stanu zalogowanego użytkownika (przez Riverpod) przebudowuje węzeł nawigacyjny `go_router` i odblokowuje dostęp do ścieżek chronionych (tzw. Route Guards).

Przepływ danych: Użytkownik wprowadza poświadczenia w UI → `AuthViewModel` pakuje dane w DTO i wywołuje `AuthRepository` → Zapytanie HTTP POST przez klienta Dio → Backend zwraca token JWT i profil pracownika → `ViewModel` parsuje odpowiedź (Either) i aktualizuje globalny `SessionProvider` → UI zostaje odświeżone.

Obsługa wyjątków: Mechanizmy walidacji początkowo sprawdzają poprawność formatu e-mail oraz wymagania dotyczące siły hasła bezpośrednio w UI. W przypadku błędu serwera (np. niepoprawne dane logowania, błąd HTTP 401), repozytorium zwraca wariant `Left` struktury `Either`. `ViewModel` przechwytuje go bez rzucania niespodziewanego wyjątku i bezpiecznie mapuje kod błędu na czytelny komunikat wyświetlany użytkownikowi za pośrednictwem natywnego okna `SnackBar`.



Rysunek 4: Widok logowania dla personelu medycznego z wbudowaną walidacją danych.

6.6.2. Moduł Rejestracji Pacjenta i Wywiadu Triage

Cel biznesowy: Bezpapierowa i zautomatyzowana procedura rejestrowania nowych przypadków w systemie SOR. Moduł dzieli się na dwie powiązane części:

1. Szybkie utworzenie kartoteki osobowej (dane demograficzne, grupa krwi, alergie).
2. Przeprowadzenie wywiadu medycznego (Triage) wraz ze zbiorem parametrów życiowych, weryfikacją bólu oraz uzyskaniem sugerowanego przez model AI stopnia pilności (KTAS).

Opis techniczny: Proces rejestracji opiera się na złożonych formularzach obsługiwanych przez `PatientFormViewModel` oraz `TriageFormViewModel`. Widoki składają się ze zoptymalizowanych widżetów formularzy, zapobiegających nadmiernemu przebudowywaniu ekranu podczas wpisywania znaków z klawiatury. Zaimplementowano moduł integracji, który po wypełnieniu niezbędnych pomiarów życiowych pozwala ratownikowi asynchronicznie przesłać wektor danych do niezależnej mikrousługi AI (FastAPI) za pomocą wstrzykniętego dostawcy usług.

Przeływ danych: Po zebraniu wywiadu, stan UI zostaje zablokowany (zabezpieczenie przed podwójnym wysłaniem - Loading State). Dane wędrują do `TriageFormViewModel`, który wywołuje najpierw API do wyznaczenia priorytetu AI. Otrzymana prognoza (od 1 do 5) jest nakładana na główny model danych i odsyłana pełnym zapytaniem POST do API Go jako nowy rekord Przyjęcia na oddział (Admission). Następuje zmiana stanu i przekierowanie z powrotem do Dashboardu.

Obsługa wyjątków: Ze względu na powagę danych medycznych, każde pole wymaga rygorystycznej, asynchronicznej walidacji (np. sprawdzanie struktury PESEL, formatowanie numerów telefonu). Jeżeli usługi AI są chwilowo niedostępne, moduł obsługi błędu (fpdart fold) przechwytuje anomalie i nie blokuje wywiadu – po prostu deaktywuje inteligentną predykcję, oddając całkowitą, wymuszoną decyzję o nadaniu priorytetu manualnego w ręce wpisującego medyka.

Rysunek 5: Interaktywny formularz Triage wspierany predykcjami uczenia maszynowego.

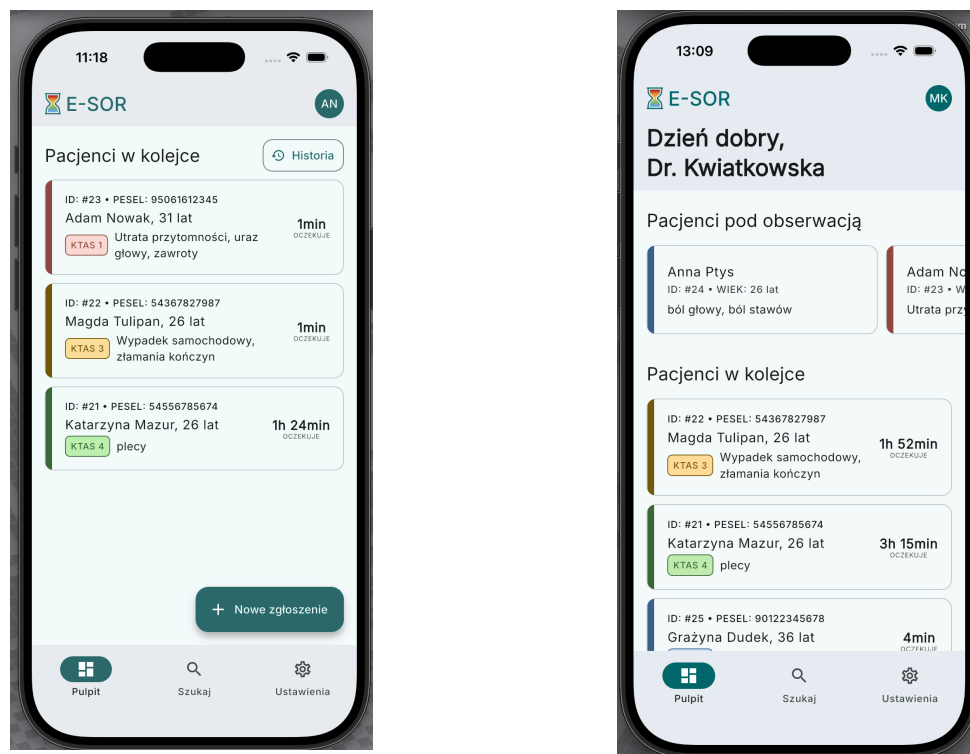
6.6.3. Moduł Dashboardu i Aktywnego Kolejowania

Cel biznesowy: Główne centrum operacyjne personelu medycznego. Ekran zapewnia błyskawiczny przegląd sytuacji na oddziale, prezentując aktualną listę pacjentów oczekujących na badanie, z wyraźnym uwzględnieniem przypisanych im priorytetów (od Kod Czerwony do Kod Niebieski).

Opis techniczny: Architektura tego widoku opiera się na ciągłym strumieniowaniu danych z użyciem protokołu WebSocket (moduł `web_socket_channel`). Dedykowana usługa `WebsocketService` podtrzymuje stałe połączenie z serwerem i konwertuje napływający strumień bajtów na listę zaktualizowanych obiektów domenowych pacjentów. Główny `ViewModel` kolejki nasłuchuje tego strumienia (za pomocą narzędzia `StreamProvider` od Riverpod) i odrysowuje kafelki pacjentów (Custom Cards). Dodatkowo zaimplementowano tu architekturę zakładek (Tabs), zintegrowaną z `go_router` – nawigacja pomiędzy główną kolejką, własnymi pacjentami w obserwacji a wyszukiwarką bazuje na zmianie ścieżek URL (`/dashboard`, `/dashboard/search` itd.), co ułatwia zarządzanie historią przeglądarki w wersji Web.

Przepływ danych: Nawiązanie połączenia Handshake z uwierzytelnieniem → Inicjalizacja `StreamProvider` → Serwer pushuje zaktualizowaną tablicę pacjentów w formacie JSON (po każdej akcji zapisu do bazy przez innego pracownika) → `ViewModel` parsuje nową ramkę → UI animuje przetasowanie się pacjentów na liście na podstawie ich pilności i czasu oczekiwania.

Obsługa wyjątków: Zerwanie połączenia WebSocket (np. wejście z tabletem w martwą strefę Wi-Fi) automatycznie wyzwała logikę ponawiania połączenia (Exponential Backoff). Podczas próby re-connectu UI wyświetla nieinwazyjny wskaźnik utraty sygnału na górnym pasku. Jeżeli pacjent zostanie obsłużony przez kogoś innego w tym samym ułamku sekundy, wyzwoli to błąd wersji zasobu przechwycony przez `fpdart`, co powiadomi użytkownika o konflikcie.



Rysunek 6: Główny ekran Dashboardu pokazujący sortowanie według kolorów Triage i czasu oczekiwania. Po prawej stronie widok dla lekarza, po lewej dla reszty personelu.

6.6.4. Moduł Konsultacji Lekarskiej (Panel Lekarza)

Cel biznesowy: Digitalizacja procesu diagnostycznego, zlecenia badań i zamknięcia procesu przyjęcia (Wypis). Pozwala lekarzowi dyżurnemu wezwać pacjenta z poczekalni (zmiana statusu), wpisać notatki z fizykalnego badania i podejmować decyzje operacyjne optymalizując rotację miejsc na oddziale.

Opis techniczny: Widok zorganizowany jest wokół kontrolera `DoctorViewModel`, wspartego o komponenty stanu z modułów współdzielonych (Shared). Implementacja zawiera interaktywne elementy zmiany statusu pacjenta (np. z „W Gabinetce” na „Oczekuje na wyniki” lub „Zakończony”). Ważnym elementem inżynierskim w tym miejscu było uniknięcie odświeżania całego widoku podczas modyfikowania jednego atrybutu (np. wpisywania notatki) poprzez użycie `ref.select()` w Riverpod. Zaimplementowano tu mechanizm opóźnionego zapisu (Debouncing) dla pól tekstowych, minimalizujący ruch sieciowy do bazy danych.

Przepływ danych: Lekarz klika wybiera pacjenta, które przyjmuje na konsultację → UI wysyła asynchroniczną komendę (Command) do `DoctorViewModel` → ViewModel strzela żądaniem PATCH do API zdejmując pacjenta z poczekalni → Serwer zwraca odpowiedź sukcesu → ViewModel zamyka okno dialogowe i wywołuje odświeżenie listy „Moich pacjentów”.

Obsługa wyjątków: Z powodu częstych przypadków niekontrolowanego zamknięcia aplikacji (np. wyładowanie tabletu), interfejs zapobiega nadpisywaniu niespójnych danych. Moduł korzysta z mechanizmu wczesnego wyjścia (Guard Clauses) i silnej walidacji – nie pozwala zamknąć

procesu leczenia pacjenta, dopóki wszystkie kroki diagnostyczne nie zostaną odznaczone. Błędy związane z przekroczeniem czasu oczekiwania serwera są łapane funkcyjnie i proponują użytkownikowi bezpieczne ponowienie akcji z zachowaniem stanu lokalnego (brak utraty wpisanego tekstu badawczego).



Rysunek 7: Interfejs aktywnej konsultacji pacjenta.

6.7. Zapewnienie Jakości Oprogramowania (Testy Jednostkowe)

W celu zagwarantowania niezawodności logiki medycznej, w projekcie wdrożono zautomatyzowane testy jednostkowe. Zgodnie z najlepszymi praktykami architektury MVVM, testowaniu podlegają wyłącznie warstwy niezależne od interfejsu graficznego (czyli ViewModel oraz modele danych domenowych).

- Do implementacji środowiska testowego wykorzystano standardowy pakiet `flutter_test`.
- Repozytoria wymieniające dane z prawdziwym API (Backendem) są symulowane w pamięci za pomocą zjawiska Mockowania (biblioteka `mocktail`).
- Takie podejście gwarantuje, że kluczowa logika decyzyjna (np. obliczanie odpowiedniego koloru Triage na podstawie czasu zgłoszenia) działa poprawnie i jest chroniona przed zjawiskiem regresji podczas rozbudowy systemu o nowe funkcje.



Rysunek 8: Wynik przeprowadzonych testów jednostkowych, jak widać ponad 70% kodu zostało objęte testami.

6.8. Budowanie, kompilacja i dystrybucja wieloplatformowa

Aplikacja kliencka E-SOR od początku była projektowana z myślą o maksymalnej elastyczności środowiskowej. Wykorzystanie silnika Flutter umożliwia kompilację tego samego kodu źródłowego (napisanego w jednym języku Dart) na zróżnicowane urządzenia końcowe, bez konieczności utrzymywania odrębnych repozytoriów dla różnych systemów:

- **Platformy Mobilne (Android/iOS):** Aplikacja może być zbudowana w formie pliku `.apk` lub zarchiwizowana do `.ipa`. Środowisko mobilne jest dedykowane przede wszystkim dla ratowników medycznych przeprowadzających Triage w biegu z użyciem przenośnych tabletów (np. iPad).
- **Platformy Desktopowe (Windows/macOS/Linux):** Wykorzystując kompilację **AOT (Ahead-of-Time)**, kod zamieniany jest na wysokowydajny, natywny plik wykonywalny maszyny (np. plik `.exe` w systemie Windows). Taka forma uruchomienia jest przeznaczona na stacjonarne stacje robocze lekarzy na SOR oraz punkty rejestracyjne.
- **Środowisko Web (Przeglądarka):** Za pomocą jednego polecenia (`build web`), projekt renderowany jest do zoptymalizowanego pakietu HTML/CSS z logiką zamienioną na kod **WebAssembly**. Umożliwia to wyświetlanie „pasywnego” widoku kolejki bezpośrednio w przeglądarce telewizora wiszącego w poczekalni (Smart TV), całkowicie omijając proces natywnej instalacji.

7. Projekt Warstwy Danych i Konfiguracja Relacyjnej Bazy Danych

Warstwa przechowywania danych w systemie E-SOR musi gwarantować bezwzględną spójność informacji, transakcyjność oraz pełne bezpieczeństwo danych historycznych pacjentów. Do realizacji tego zadania wybrano relacyjny system zarządzania bazą danych (RDBMS) **PostgreSQL 15**, współpracujący z systemem backendowym za pośrednictwem biblioteki ORM **GORM**.

7.1. Uzasadnienie wyboru bazy danych PostgreSQL

Wybór PostgreSQL jako centralnego repozytorium danych podyktowany był kluczowymi cechami tego systemu w kontekście architektury systemów medycznych:

- **Pełna zgodność z ACID:** Gwarantuje, że każda operacja (np. jednoczesne przypisanie pacjenta do gabinetu przez lekarza i zmiana statusu kolejki przez ratownika) zostanie wykonana w sposób atomowy, spójny, odizolowany i trwały, co zapobiega powstawaniu anomalii w bazie.
- **Zaawansowana obsługa typów wyliczeniowych (ENUM):** Pozwala na natywne definiowanie stanów w bazie danych (np. kolory KTAS, role personelu), co drastycznie zwiększa integralność danych na poziomie silnika bazy.
- **Wydaźność indeksowania:** PostgreSQL oferuje zaawansowane mechanizmy indeksowania (B-Tree, GIN), co pozwala na natychmiastowe przeszukiwanie bazy po numerach PESEL lub fragmentach nazwisk pacjentów przy użyciu operatorów **ILIKE**.

7.2. Architektura mapowania obiektowo-relacyjnego (GORM & Auto-Migracje)

Tradycyjne podejście do zarządzania schematem bazy danych za pomocą manualnych skryptów `.sql` w zespole wieloosobowym często prowadzi do konfliktów strukturalnych. Aby temu zapobiec, wdrożono standard **Code-First** przy użyciu biblioteki **GORM**.

Mechanizm Auto-Migracji: Podczas każdego uruchomienia kontenera `sor_backend`, jądro aplikacji w Go wykonuje funkcję `db.AutoMigrate()`. GORM analizuje zdefiniowane w kodzie struktury (`structs`), porównuje je z aktualnym schematem w PostgreSQL i automatycznie:

1. Tworzy brakujące tabele.
2. Dopisuje nowe kolumny lub modyfikuje typy danych, jeśli uległy zmianie w kodzie.
3. Tworzy klucze obce (**Foreign Keys**) oraz tabele asocjacyjne dla relacji wiele-do-wielu.

Proces ten jest w 100% bezdestrukcyjny dla istniejących już rekordów, co drastycznie przyspiesza proces wdrażania zmian w zespole deweloperskim oraz na serwerze produkcyjnym za pomocą potoku CI/CD.

7.3. Struktura tabel oraz relacji

Na podstawie założeń biznesowych oraz struktury relacyjnej zaimplementowano fizyczny model bazy danych w systemie PostgreSQL. Poniżej przedstawiono specyfikację słowników systemowych (typów wyliczeniowych) oraz strukturę poszczególnych tabel wraz z typami danych i więzami integralności.

7.3.1. Słowniki systemowe (Typy ENUM)

W celu zapewnienia spójności danych i eliminacji błędów typu literowego na poziomie silnika bazy danych, zdefiniowano następujące typy wyliczeniowe:

- gender_enum: ('M', 'K', 'INNY') — Płeć pacjenta.
- blood_group_enum: ('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', '0+', '0-') — Grupa krwi.
- role_enum: ('PIELEGNIAK', 'RATOWNIK', 'LEKARZ', 'ADMIN') — Uprawnienia personelu.
- admission_status_enum: ('W_POCZEKALNI', 'W_GABINECIE', 'OCZEKUJE_NA_WYNIKI', 'ZAKONCZONE') — Stan pacjenta na SOR.

7.3.2. Specyfikacja tabel bazodanowych

7.3.2.1. Tabela: patients (Pacjenci)

Nazwa kolumny	Typ danych	Ograniczenia	Opis i przeznaczenie
id	SERIAL	PRIMARY KEY	Unikalny identyfikator pacjenta
pesel	VARCHAR(11)	UNIQUE, NOT NULL	Numer PESEL walidowany systemowo
first_name	VARCHAR(50)	NOT NULL	Imię pacjenta
last_name	VARCHAR(50)	NOT NULL	Nazwisko pacjenta
date_of_birth	TIMESTAMP	NOT NULL	Data urodzenia
gender	gender_enum	NOT NULL	Płeć (słownik)
address	TEXT	NOT NULL	Adres zamieszkania
phone	VARCHAR(20)	NOT NULL	Telefon kontaktowy
email	VARCHAR(100)		Adres e-mail (opcjonalny)
emergency_contact_name	VARCHAR(100)	NOT NULL	Imię i nazwisko kontaktu alarmowego
emergency_contact_phone	VARCHAR(20)	NOT NULL	Telefon kontaktu alarmowego
blood_group	blood_group_enum		Grupa krwi (opcjonalna)
allergies	TEXT		Alergie i uczulenia (wywiad)
chronic_diseases	TEXT		Choroby przewlekłe pacjenta
created_at	TIMESTAMP	DEFAULT NOW	Znacznik czasu rejestracji w systemie

7.3.2.2. Tabela: staff (Personel)

Nazwa kolumny	Typ danych	Ograniczenia	Opis i przeznaczenie
id	SERIAL	PRIMARY KEY	Unikalny identyfikator pracownika
first_name	VARCHAR(50)	NOT NULL	Imię pracownika
last_name	VARCHAR(50)	NOT NULL	Nazwisko pracownika
academic_title	VARCHAR(30)		Tytuł naukowy / zawodowy
role	role_enum	NOT NULL	Rola i poziom uprawnień w systemie

login_email	VARCHAR(100)	UNIQUE, NOT NULL	Adres e-mail do logowania
password_hash	VARCHAR(255)	NOT NULL	Zabezpieczony skrót hasła (Bcrypt)
created_at	TIMESTAMP	DEFAULT NOW	Data utworzenia konta

7.3.2.3. Tabela: admissions (Przyjęcia i Triage na SOR)

Nazwa kolumny	Typ danych	Ograniczenia	Opis i przeznaczenie
id	SERIAL	PRIMARY KEY	Numer zgłoszenia / karty przyjęcia
id_pacjenta	INTEGER	FOREIGN KEY, NOT NULL	Identyfikator pacjenta
id_osoby_przyjmujacej	INTEGER	FOREIGN KEY, NOT NULL	Pracownik wykonujący Triage
id_lekarza_prowadzacego	INTEGER	FOREIGN KEY, NOT NULL	Lekarz prowadzący badanie
data_przyjecia	TIMESTAMP	DEFAULT NOW	Fizyczny czas rejestracji na SOR
forma_przybycia	VARCHAR(100)	NOT NULL	Tryb transportu (Słownik Go)
injury	BOOLEAN	DEFAULT FALSE	Czy pacjent z urazem fizycznym
mental_status	VARCHAR(100)	NOT NULL	Stan psychiczny / świadomość (Słownik)
pain	BOOLEAN	DEFAULT FALSE	Czy pacjent zgłasza ból
pain_lvl	INTEGER	NOT NULL	Poziom bólu
hr	INTEGER	NOT NULL	Tętno (Heart Rate)
sbp	INTEGER	NOT NULL	Ciśnienie skurczowe
dbp	INTEGER	NOT NULL	Ciśnienie rozkurczowe
rr	INTEGER	NOT NULL	Częstość oddechów (Respiratory Rate)
bt	NUMERIC(3,1)	NOT NULL	Temperatura ciała
chief_complaint	TEXT	NOT NULL	Główny powód zgłoszenia
priority_ktas	INTEGER	NOT NULL	Kod pilności w klasyfikacji KTAS (1-5)

is_ai_predicted	BOOLEAN	DEFAULT FALSE	Czy priorytet wyliczył model AI
status_przyjecia	admission_status_enum	DEFAULT 'W_AROZNEKAPN'	Artykuł o obsłudze pacjenta

7.3.2.4. Tabela: consultations (Konsultacje Lekarskie)

Nazwa kolumny	Typ danych	Ograniczenia	Opis i przeznaczenie
id	SERIAL	PRIMARY KEY	Identyfikator konsultacji
id_przyjecia	INTEGER	FOREIGN KEY, NOT NULL	Powiązane ID przyjęcia z karty
id_lekarza	INTEGER	FOREIGN KEY, NOT NULL	Lekarz przeprowadzający badanie
wywiad_lekarski	TEXT	NOT NULL	Badanie podmiotowe i przedmiotowe
rozpoznanie_icd10	TEXT	NOT NULL	Ostateczna diagnoza lub kod ICD-10
decyzja_wyjsciowa	VARCHAR(100)	NOT NULL	Decyzja kończąca pobyt na SOR
data_konsultacji	TIMESTAMP	DEFAULT NOW	Znacznik czasu zamknięcia konsultacji

7.3.2.5. Tabela: diagnostic_orders (Zlecenia Diagnostyczne)

Nazwa kolumny	Typ danych	Ograniczenia	Opis i przeznaczenie
id	SERIAL	PRIMARY KEY	Unikalny numer zlecenia
id_przyjecia	INTEGER	FOREIGN KEY, NOT NULL	Powiązany identyfikator karty przyjęcia
id_lekarza	INTEGER	FOREIGN KEY, NOT NULL	Lekarz zlecający badanie
typ_badania	VARCHAR(100)	NOT NULL	Rodzaj badania diagnostycznego (np. RTG, KREW)
opis_zlecenia	TEXT		Dodatkowe instrukcje od lekarza
status_badania	VARCHAR(100)	DEFAULT 'ZLECONIE'	Aktualny status realizacji badania

data_zlecenia	TIMESTAMP	DEFAULT NOW	Znacznik czasu zlecenia procedury
---------------	-----------	----------------	-----------------------------------

7.3.3. Relacje i więzy integralności referencyjnej

Integralność strukturalna bazy danych chroniona jest za pomocą kluczy obcych (Foreign Keys) z ostro zdefiniowanymi regułami kaskadowości, co zapobiega powstawaniu tzw. rekordów osieroconych.

7.3.4. Integralność referencyjna i relacje bazodanowe

W celu zapewnienia pełnej spójności danych oraz automatyzacji procesów czyszczenia i zabezpieczania rejestrów medycznych, w relacyjnej bazie danych PostgreSQL zaimplementowano następujące powiązania kluczy obcych:

1. Relacja **patients** → **admissions** (Jeden-do-Wielu):

- Powiązanie przez: `admissions.id_pacjenta` odwołuje się bezpośrednio do `patients.id`.
- Reguła klucza obcego: `ON DELETE RESTRICT`. Blokuje możliwość usunięcia kartoteki pacjenta z bazy danych, jeżeli posiada on przypisaną jakąkolwiek aktywną lub historyczną kartę przyjęcia na SOR-ze.

2. Relacja **staff** → **admissions** (Jeden-do-Wielu):

- Powiązanie przez: `admissions.id_osoby_przyjmujacej` (personel wykonujący Triage) oraz `admissions.id_lekarza_prowadzacego` (przypisany lekarz prowadzący).
- Reguła dla Triage: `ON DELETE RESTRICT`. Uniemożliwia usunięcie konta pracownika medycznego z bazy, jeżeli podpisał on kartę segregacji medycznej.
- Reguła dla Lekarza: `ON DELETE SET NULL`. W przypadku usunięcia konta lekarza ze struktury szpitala, pole lekarza prowadzącego w karcie przyjęcia staje się puste (`NULL`), co pozwala na bezproblemowe przypisanie pacjenta do innego członka personelu.

3. Relacja **admissions** → **consultations** (Jeden-do-Jednego):

- Powiązanie przez: `consultations.id_przyjecia` odwołuje się do `admissions.id` z nałożonym ograniczeniem unikalności `UNIQUE`.
- Reguła klucza obcego: `ON DELETE CASCADE`. Usunięcie rekordu przyjęcia medycznego z systemu automatycznie kasuje powiązaną z nim kartę konsultacji i diagnozy lekarskiej.

4. Relacja **admissions** → **diagnostic_orders** (Jeden-do-Wielu):

- Powiązanie przez: `diagnostic_orders.id_przyjecia` odwołuje się do `admissions.id`.
- Reguła klucza obcego: `ON DELETE CASCADE`. Skasowanie z bazy danych karty przyjęcia usuwa synchronicznie całą powiązaną historię zleceń na badania laboratoryjne oraz obrazowe (np. RTG, TK, EKG).

7.3.5. Przykładowe zapytania SQL używane w aplikacji

Poniższa sekcja przedstawia wybrane, rzeczywiste zapytania SQL generowane przez warstwę dostępu do danych (GORM) i wykonywane na bazie danych PostgreSQL w trakcie kluczowych operacji systemowych.

7.3.5.1. 1. Pobranie aktywnej kolejki pacjentów (Preload i sortowanie KTAS)

Zapytanie pobiera pacjentów w poczekalni, automatycznie dociągając powiązane dane osobowe w ramach jednego kroku transakcyjnego:

```
SELECT admissions.*, patients.*
FROM admissions
LEFT JOIN patients ON admissions.id_pacjenta = patients.id
WHERE admissions.status_przyjecia = 'W_POCZEKALNI'
ORDER BY admissions.priority_ktas ASC, admissions.data_przyjecia ASC;
```

7.3.5.2. 2. Pobranie szczegółów pojedynczego przyjęcia po ID

Zapytanie wykorzystywane przez lekarza w gabinecie do załadowania pełnej karty medycznej konkretnego zgłoszenia:

```
SELECT * FROM admissions WHERE admissions.id = 1 LIMIT 1;
SELECT * FROM patients WHERE patients.id = 1;
```

7.3.5.3. 3. Filtrowanie archiwum przyjęć na SOR

Zapytanie dynamiczne z uwzględnieniem parametrów filtrowania przekazywanych przez personel (np. filtrowanie po stanie „ZAKONCZONE” i kodzie KTAS 3):

```
SELECT admissions.* FROM admissions
WHERE admissions.status_przyjecia = 'ZAKONCZONE' AND admissions.priority_ktas =
3
ORDER BY admissions.data_przyjecia DESC;
```

7.3.5.4. 4. Zapisanie nowej karty Triage (Rejestracja)

Zapytanie typu INSERT zapisujące parametry życiowe wprowadzone przez ratownika medycznego z jawnym pominięciem nadpisywania powiązanych tabel asocjacyjnych:

```
INSERT INTO admissions (
  id_pacjenta, id_osoby_przyjmujacej, id_lekarza_prowadzacego, data_przyjecia,
  forma_przybycia, injury, mental_status, pain, pain_lvl, hr, sbp, dbp, rr, bt,
  chief_complaint, priority_ktas, is_ai_predicted, status_przyjecia
) VALUES (
  1, 2, NULL, CURRENT_TIMESTAMP, 'Karetka publiczna', true,
  'Nieprzytomny/Brak reakcji', true, 9, 145, 85, 50, 24, 36.2,
  'Pacjent po wypadku samochodowym...', 1, true, 'W_POCZEKALNI'
) RETURNING id;
```

7.3.5.5. 5. Aktualizacja statusu pacjenta (Wezwanie do gabinetu)

Zapytanie modyfikujące stan procesu obsługi pacjenta na podstawie akcji lekarza bądź pielęgniarki:

```
UPDATE admissions
SET status_przyjecia = 'W_GABINECIE'
WHERE id = 1;
```

7.4. Diagram UML bazy danych

7.4.1. Generowanie Diagramu UML Struktury Bazy Danych

W celu zweryfikowania spójności relacji oraz stworzenia czytelnej, graficznej dokumentacji architektury danych systemu E-SOR, wykorzystano deklaratywne narzędzie **Mermaid**. Podejście to pozwoliło na odejście od klasycznych, manualnych programów graficznych na rzecz standardu *Diagram-as-Code* (diagram jako kod).

7.4.1.1. Metodologia deklaratywna (Diagram-as-Code)

Proces tworzenia schematu UML (Entity-Relationship / Class Diagram) polegał na bezpośrednim zmapowaniu struktur tabel zdefiniowanych w skrypcie implementacyjnym bazy danych PostgreSQL na tekstową składnię Mermaid. Wykorzystanie tego rozwiązania przyniosło następujące korzyści inżynierskie:

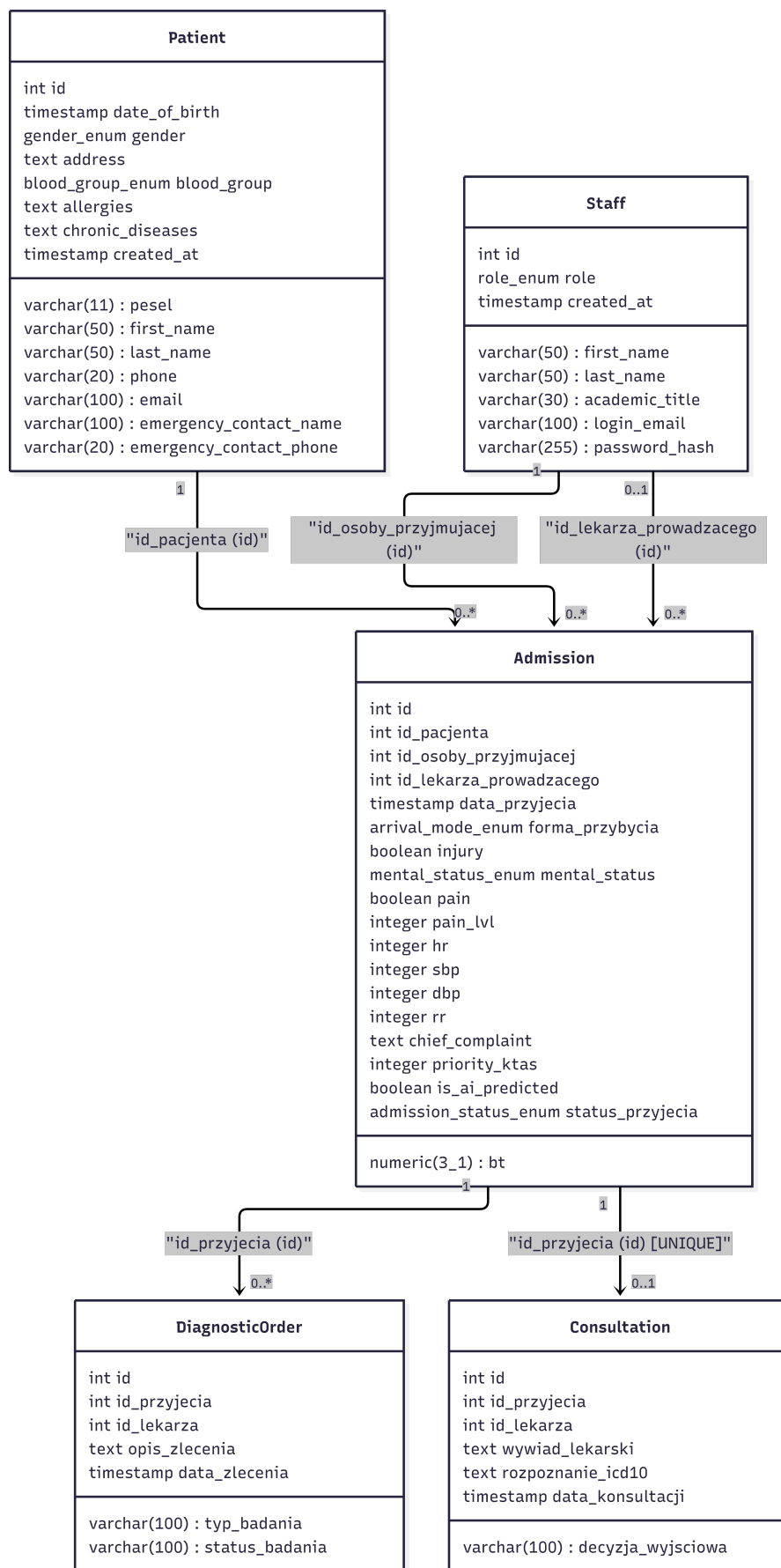
- **Pełna spójność:** Pola klas, typy danych (w tym niestandardowe systemowe typy `ENUM`), krotności relacji oraz nazwy kluczy obcych odpowiadają w skali 1:1 fizycznej strukturze bazy danych.
- **Łatwość utrzymania:** Każda przyszła modyfikacja schematu bazy (np. dodanie kolumny czy zmiana relacji) wymaga jedynie edycji kilku linii tekstu czytelnej składni Mermaid, eliminując potrzebę ręcznego przesuwania bloków i strzałek na schemacie graficznym.
- **Optymalizacja układu:** Dzięki wykorzystaniu silnika renderującego `layout: elk` w konfiguracji diagramu, proces rozmieszczenia klas na płaszczyźnie, trasowanie linii powiązań oraz unikanie krzyżowania się strzałek relacji zostało w pełni zautomatyzowane i zoptymalizowane algorytmicznie.

7.4.1.2. Środowisko i narzędzia renderujące

Tekstowy kod źródłowy diagramu został przetworzony i wyrenderowany przy użyciu darmowego, otwartoźródłowego środowiska **Mermaid Live Editor**. Narzędzie to umożliwiło podgląd struktury w czasie rzeczywistym oraz walidację poprawności składniowej asocjacji.

Gotowy, w pełni zoptymalizowany pod kątem czytelności schemat graficzny został wyeksportowany do pliku wektorowego (SVG) oraz rastrowego o wysokiej rozdzielczości (PNG). Pozwoliło to na bezproblemowe osadzenie diagramu w głównym pliku dokumentacji systemu bez ryzyka utraty ostrości tekstu i linii podczas końcowego renderowania dokumentu do formatu PDF.

7.5. Diagram UML



Rysunek 9: Graficzny schemat klas i relacji (UML ERD) bazy danych systemu E-SOR

8. Podsumowanie i Wnioski Końcowe

Niniejsza dokumentacja projektowa przedstawiła proces projektowania, implementacji oraz wdrażania rozproszonego systemu wspomagania klasyfikacji priorytetów pacjentów E-SOR. Realizacja projektu pozwoliła na stworzenie w pełni funkcjonalnego, bezpiecznego i wysoko-wydajnego ekosystemu informatycznego, który skutecznie odpowiada na realne problemy współczesnej medycyny ratunkowej, takie jak przełudnienie oddziałów ratunkowych oraz subiektywizm podczas manualnej oceny stanu zdrowia pacjentów.

8.1. Stopień realizacji celów inżynierskich

Wszystkie założenia techniczne i merytoryczne postawione przed systemem E-SOR zostały pomyślnie zrealizowane:

- **Wydajność warstwy serwerowej:** Wykorzystanie języka Go oraz frameworku Gin Gonic pozwoliło na osiągnięcie znikomego narzutu pamięciowego (4-12 MB RAM w stanie spoczynku) oraz natychmiastowego czasu reakcji. Zastosowanie Goroutines i planera Go Scheduler umożliwiło stabilną, masową obsługę otwartych socketów TCP bez blokowania wątków systemowych.
- **Reaktywność w czasie rzeczywistym:** Implementacja dwukierunkowej komunikacji opartej na protokole WebSocket wyeliminowała konieczność zasobożernego odpytywania serwera (polling). Backend synchronicznie reaguje na każdą mutację stanu bazy danych i natychmiastowo rozgłasza (broadcast) zaktualizowaną strukturę kolejki do podłączonych klientów medycznych oraz anonimowego strumienia telewizora poczekalni (zgodnego z RODO).
- **Autonomia modułu analitycznego AI:** Autorska klasa drzewa decyzyjnego, zintegrowana asynchronicznym frameworkiem FastAPI, realizuje inferencję matematyczną w czasie mierzonym w milisekundach. Mimo ogólnej dokładności na poziomie 52%, model osiągnął klinicznie doskonałą czułość ($\text{Recall} = 1.00$) dla pacjentów w stanie krytycznym, co gwarantuje pełne bezpieczeństwo medyczne poprzez bezbłędne wychwytywanie stanów zagrożenia życia.
- **Bezpieczeństwo i konteneryzacja DevOps:** Całość systemu została skutecznie odizolowana środowiskowo przy użyciu technologii Docker i orkiestrowana przez Docker Compose. Prywatność danych medycznych oraz bezpieczną łączność zdalną serwera działającego w architekturze Homelab zapewniono poprzez wdrożenie szyfrowanej sieci kratowej VPN (Tailscale/WireGuard) z restrykcyjną polityką autoryzacji urządzeń WAN.

8.2. Wnioski deweloperskie i edukacyjne

Z perspektywy zespołu programistycznego, kluczowym aspektem projektu był jego walor edukacyjny. Podjęcie decyzji o rezygnacji z automatycznych generatorów kodu na rzecz 100% manualnego wytworzenia struktur aplikacyjnych wymusiło dogłębne zrozumienie cyklu życia żądań HTTP, mechaniki mapowania obiektowo-relacyjnego (GORM) oraz niskopoziomowej manipulacji przestrzeniami nazw w środowisku Python (pickle/sys.modules).

Wybór silnie typowanego, kompilowanego języka Go pozwolił deweloperom na poszerzenie kompetencji inżynierskich poza standardowy program studiów akademickich, udowadniając przewagę technologiczną rozwiązań natywnych w systemach dedykowanych dla infrastruktury o ograniczonych zasobach sprzętowych.

8.3. Kierunki dalszego rozwoju systemu

Pomimo osiągnięcia pełnej gotowości produkcyjnej systemu w wersji MVP, zidentyfikowano następujące obszary, które mogą stanowić fundament pod dalszy rozwój projektu (np. w ramach pracy magisterskiej):

1. **Rozbudowa wektora cech AI:** Przejście z prostego modelu drzewa decyzyjnego na zaawansowane algorytmy ansamblowe (np. Random Forest lub XGBoost) oraz rozszerzenie wektora wejściowego o parametry biochemiczne krwi lub wyniki wstępnych badań laboratoryjnych w celu zwiększenia celności klasyfikacji stanów pośrednich (KTAS 2 i 3).
2. **Wdrożenie pełnego potoku CI/CD:** Automatyzacja testów jednostkowych i integracyjnych po stronie repozytorium GitHub wraz z automatycznym wdrażaniem (deployment) obrazów na serwer produkcyjny za pomocą mechanizmu Webhooków.
3. **Integracja z systemami zewnętrznymi:** Dostosowanie interfejsu API do standardu HL7 FHIR (Fast Healthcare Interoperability Resources), co umożliwiłoby pełną, natywną wymianę danych z centralnymi systemami szpitalnymi (HIS) oraz ogólnokrajową siecią ratownictwa medycznego.